Doctoral Dissertation

An Intent-Aware and Transaction-Independent Source-Level Debugging Assistant for Solidity Smart Contracts

Inseong Jeon

Department of Computer Science

Graduate School

Korea University

February 2027

An Intent-Aware and Transaction-Independent Source-Level Debugging Assistant for Solidity Smart Contracts

by
Inseong Jeon

---

under the supervision of Professor Hoh Peter In

A dissertation submitted in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy

Department of Computer Science

Graduate School
Korea University

October 2026

The dissertation of Inseong Jeon has been approved
by the dissertation committee in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy

December 2026

_____

Committee Chair:

_____

Committee Member:

_____

Committee Member:

_____

Committee Member:

_____

Committee Member:

An Intent-Aware and Transaction-Independent Source-Level Debugging Assistant for Solidity Smart Contracts

Inseong Jeon
Department of Computer Science
under the supervision of Professor Hoh Peter In

Abstract

TODO: English abstract.

Keywords: smart contract; Solidity; developer intent; abstract interpretation; interval domain; incremental analysis; debugging

국문 제목

전 인 성
컴 퓨 터 학 과
지 도 교 수: 인 호

국문 초록

TODO: 국문 초록.

중심어 : 스마트 컨트랙트; 솔리디티; 개발자 의도; 추상해석; 구간 도메인; 증분 분석; 디버깅

TODO: dedication (optional; no heading, centered).

Preface

TODO: preface. It must state, at minimum:

1. which results were obtained in collaboration, the nature and proportion of others' contributions, and which portions are claimed as original;
2. funding sources and other contributors;
3. which contents have been published or submitted for publication, and the contributions of every author of those multi-authored works (SOLQDEBUG, INTENTCHECKER);
4. optionally: background, motivation, target readership.

Acknowledgment

TODO: acknowledgment.

Table of Contents

Abstract i

국문초록 ii

Preface iv

Acknowledgment v

Table of Contents vi

List of Tables ix

List of Figures x

List of Listings xii

Nomenclature xiii

1 Introduction 1

1.1 Motivation 1

1.2 Proposed Techniques 3

1.3 Contributions 4

1.4 Outlines 5

2 Background 6

2.1 Solidity Execution Model and Debugging Workflow 6

2.2 Numeric Logic Errors in Solidity 9

3 Related Work **13**

3.1 Solidity Debugging and Development Environments 13
3.2 Numeric Logic Error Detection in Solidity 14
3.3 Specification-Based Smart Contract Analysis 15
3.4 Abstract Interpretation and Interactive Static Analysis 16

4 The Design of IQDebug **17**

4.1 System Architecture 17
4.2 Source-Level Program Analysis 19
4.2.1 Source-Level Program Representation 20
4.2.2 Debugging Input Model 30
4.2.3 Abstract Interpretation of Program States 32
4.3 Intent-Aware Validation 36
4.4 Source-Level Debugging over Abstract Program States 53
4.4.1 Forward and Backward Navigation 55
4.4.2 Function-Call Navigation 56
4.4.3 Breakpoint-Based Navigation 57
4.4.4 Navigation over Control-Flow Structures 58

5 Evaluation **60**

5.1 Experimental Setup 60
5.2 Intent Validation Capability 60
5.3 Specification and Analysis Cost 60
5.3.1 Specification Cost 60
5.3.2 Validation and Interactive Analysis Cost 60
5.4 Comparison with Existing Detection Tools 60
5.5 Threats to Validity 60

6 Discussion **61**

6.1 Design Trade-offs of Source-Level Debugging 61
6.2 Practical Guidelines for Debug Inputs and Intent Specification 61

6.3 Current Limitations 61

7 Conclusion **62**

Bibliography **63**

A Incremental CFG Construction Patterns **67**

A.1 CFG Hierarchy 68

A.2 Statement-Level Construction Patterns 69

B Abstract Domain and Formal Semantics **74**

B.1 Language Syntax 74

B.2 Abstract Domain 75

B.3 Concrete Semantics 77

B.4 Collecting Semantics 77

B.4.1 Abstract Semantics 78

List of Tables

2.1 Representative single-transaction numeric logic error patterns from prior work, classified by abstraction level. 11

4.1 Interactive parser entry rules 21
4.2 Line-to-node index mapping by statement type 24
4.3 Fields of the validation context $\Gamma \in \mathbb{C}$. Each intent form reads only the fields required by its semantics. 38
4.4 Per-operator `Satisfied`/`Violated` semantic conditions. 49
4.5 Per-operator violated-region decomposition. 51

B.4 Concrete semantics (denotational) 80
B.5 Abstract semantics (denotational) 81

List of Figures

1.1 Example requiring state preparation before debugging the target function. 2

2.1 Compilation in a conventional Solidity debugging workflow. 7
2.2 Contract deployment in a conventional Solidity debugging workflow. . . . 7
2.3 State initialization and target-function execution in a conventional Solidity debugging workflow. 8
2.4 Transaction-based bytecode debugging in a conventional Solidity debugging workflow. 9

4.1 Overall architecture of IQDEBUG. 18
4.2 Grammar of the debugging input. 31
4.3 Grammar of developer-authored intent. 40
4.4 Spectrum of L vs R configurations for $op =$ “$>$”. 50

A.1 Structure of ContractCFG. The contract-level CFG sequences state variable initialization, constructor execution, and all function definitions. . . . 68
A.2 Structure of FunctionCFG. Each function maintains its own control-flow graph. When modifiers are present, control flows from the function entry to the modifier, and returns from the modifier's placeholder (_) back to the function body, eventually reaching the function exit. 69
A.3 Simple statement insertion: single node spliced between current node and successors 69
A.4 If statement insertion. The builder creates a `condition node`, two nodes for true/false arms, and an `if join` 70

A.5 Else-if statement insertion. The builder replaces the false arm with a new `condition node`, two nodes, and an `else-if join` 70
A.6 Else statement insertion. The builder replaces the false branch node with a new else node, connecting to the `if join` 71
A.7 While loop insertion. The builder creates a `fixpoint evaluation node` ϕ, a `condition node`, a loop body node, and a `loop exit node` . 71
A.8 Break statement insertion. The `break` node's outgoing edge is redirected to the `loop exit node` 72
A.9 Continue statement insertion. The `continue` node's outgoing edge is redirected to the loop's `fixpoint evaluation node` ϕ 72
A.10 Return statement insertion. The containing node's edge is rewired to the function's unique `return exit node` 73
A.11 Require/assert statement insertion. The builder creates a `condition node` with true edge to a node and false edge to the `error exit node` . . 73
A.12 Revert statement insertion. The containing node's edge is rewired to the function's unique `error exit node` 73

List of Listings

Nomenclature

Intv	interval abstract domain
Store	abstract store (variable $\mapsto$ abstract value)
Env	environment
$\sqcup$	join (least upper bound)
$\sqcap$	meet (greatest lower bound)
∇	widening operator
$\triangle$	narrowing operator
$[\![\cdot]\!]^\sharp$	abstract semantics (transfer function)
$\top$, $\bot$	top and bottom of the abstract domain

Abbreviation	
CFG	Control-Flow Graph
SESE	Single-Entry Single-Exit (region)
AI	Abstract Interpretation
IR	Intermediate Representation

Chapter 1. Introduction

1.1 Motivation

Blockchain technology has evolved beyond cryptocurrency into a platform for decentralized applications. Smart contracts form a core component of these applications, with Solidity serving as one of the primary languages for implementing contract logic on Ethereum [23]. As smart contracts become more complex and increasingly manage financially significant states and assets, understanding and debugging their behavior during development becomes increasingly important. In particular, Solidity programs frequently manipulate numerical state such as balances, rewards, exchange amounts, reserves, and interest-related values. Our analysis of 101 smart contracts from DAppSCAN-source [14, 36], covering 1,023 non-trivial functions, found that 82% of the functions use numeric types. Incorrect computations over these values may lead to unintended contract states even when execution completes successfully.

This dissertation focuses on two limitations that hinder effective debugging and analysis support in current Solidity development workflows.

Problem Definition 1: Transaction-Dependent and Bytecode-Dependent Debugging. Conventional Solidity debugging is tightly coupled to concrete transaction execution. To inspect the behavior of a target function, developers typically compile and deploy the contract, prepare an appropriate contract state, execute the target transaction with concrete inputs, and then inspect the resulting execution trace. If the target function depends on previously established balances, counters, reserves, permissions, or other persistent states, additional transactions may be required before the relevant program behavior can be reached. Exploring different input conditions or state configurations consequently re-

```
contract Example {
mapping(address => uint256) balances;

function deposit() external payable {
    balances[msg.sender] += msg.value;
}

function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount);
    balances[msg.sender] -= amount;
    payable(msg.sender).transfer(amount);
}
```

Figure 1.1: Example requiring state preparation before debugging the target function.

quires repeated preparation and execution of concrete debugging scenarios.

Figure 1.1 illustrates a simple example. The `withdraw` function can proceed beyond the `require` condition only when the caller has a sufficient balance. Therefore, a developer who wants to inspect the balance-update statement must first execute `deposit` to establish an appropriate contract state before invoking `withdraw`. If the debugging scenario changes, this preparation may need to be repeated. Similarly, inspecting the function under different input conditions or state configurations requires additional concrete executions. Because each execution represents only one particular combination of inputs and contract state, iterative debugging may repeatedly require developers to reconstruct suitable execution contexts.

Moreover, detailed debugging information is commonly derived from the executed EVM bytecode. Although such information accurately reflects a particular transaction, developers write and reason about contracts primarily at the Solidity source-code level. A single source-level statement may correspond to multiple EVM instructions, making it difficult to directly follow how source-level variables and numerical values evolve through the program. Accordingly, conventional debugging provides limited support for directly inspecting possible source-level behavior during development without repeatedly constructing concrete execution contexts.

Problem Definition 2: Limited Support for Developer-Authored Intent. The limitations of conventional debugging become particularly important for numeric logic errors. Unlike failures that cause compilation errors or transaction reverts, numeric logic errors

may allow execution to complete normally while violating numerical relationships intended by the developer. Correctness in such cases depends on whether the resulting numerical behavior conforms to contract-specific relationships intended by the developer.

Existing smart-contract analyses detect many classes of vulnerabilities and logic errors using predefined vulnerability patterns, type constraints, arithmetic rules, inferred properties, or previously characterized bug scenarios. Such criteria can identify defects that correspond to properties represented by the analysis. However, existing analysis workflows generally provide limited support for developers to directly supply contract-specific numerical intent as an analysis criterion. Consequently, program behavior may satisfy the generic criteria recognized by an analysis tool while still violating the numerical relationship intended for a particular contract.

1.2 Proposed Techniques

To address these two problems, this dissertation presents IQDebug, an intent-aware responsive debugging assistant for Solidity. IQDebug addresses the two problem definitions through the following two complementary solutions.

Proposed Solution 1: Transaction-Independent Source-Level Debugging. IQDebug analyzes Solidity source code using abstract interpretation rather than relying exclusively on concrete transaction execution. Specifically, developers provide debugging inputs that specify value ranges for program variables relevant to the debugging scenario, and these ranges are represented internally as abstract values in the interval domain. The analysis propagates these abstract values through the source-level control-flow graph and computes the abstract state of program variables at individual program points. Because a single abstract state can represent multiple concrete executions, developers can inspect how variable values may evolve across Solidity source-level statements without repeatedly preparing and executing every corresponding transaction.

In addition, the abstract states computed at individual program points are stored in the source-level control-flow graph, enabling developers to debug program behavior directly at the Solidity source-code level. Developers can step through Solidity statements, navigate into and out of function calls, explore feasible control-flow branches, and move between source-level breakpoints while inspecting the abstract state associated with each

program point.

Proposed Solution 2: Development-Time Intent Validation. Source-level abstract states describe how program values may evolve, but they do not by themselves determine whether the resulting behavior is correct. IQDebug therefore allows developers to explicitly specify numerical relationships that represent their intended program behavior and evaluates the specified intent over relevant source-level program states to determine whether the analyzed behavior satisfies, violates, or cannot conclusively establish the intended numerical relationship. The validation result is integrated into the source-level debugging context, allowing developers to inspect both the evolution of abstract values and their consistency with the specified intent while navigating through the source code.

These two solutions directly address the problems identified in Section 1.1. Transaction-independent source-level debugging addresses the dependence on repeated concrete transaction execution and bytecode-level inspection by enabling developers to examine abstract program states directly at Solidity source-level program points. Development-time intent validation addresses the limited support for developer-authored analysis criteria by allowing developers to explicitly specify numerical relationships and validate abstract program behavior against them. By integrating these two solutions, IQDebug supports source-level debugging of Solidity programs together with intent-aware validation of numerical behavior during development.

1.3 Contributions

The main contributions of this dissertation are summarized as follows.

- **Transaction-independent source-level debugging for Solidity.** This dissertation presents a debugging approach that analyzes Solidity source code without requiring a separate concrete transaction execution for every debugging scenario. Using abstract interpretation over an interval domain, the approach computes program-point abstract states and enables developers to inspect how variable values may evolve directly at the Solidity source-code level.
- **Developer-authored intent validation over abstract program states.** This dissertation introduces a mechanism that allows developers to explicitly specify nu-

merical relationships representing intended program behavior. These intents are evaluated over abstract program states and provide contract-specific analysis criteria for determining whether the analyzed behavior satisfies, violates, or cannot conclusively establish the specified numerical relationship.

- **Integration of source-level debugging and intent-aware validation.** This dissertation integrates source-level abstract debugging and developer-authored intent validation into a unified debugging workflow. The resulting system allows developers to navigate source-level control flow while inspecting abstract variable states together with associated intent-validation results.

- **Empirical evaluation of capability, cost, and comparison with existing approaches.** This dissertation evaluates the proposed approach in terms of intent expressibility and validation capability, specification and analysis cost, and comparison with existing numeric logic error detection tools. The evaluation also measures the computational cost of analysis and interactive updates in the integrated debugging environment.

1.4 Outlines

The remainder of this dissertation is organized as follows. Chapter 2 presents the background on Solidity debugging workflows, and numeric logic errors. Chapter 3 reviews related work on Solidity debugging, specification-based smart contract analysis, and abstract interpretation for interactive static analysis. Chapter 4 describes the design of IQDebug, including source-level program analysis, intent-aware validation, and source-level debugging over abstract program states. Chapter 5 evaluates IQDebug in terms of validation capability, specification and analysis cost, and comparison with existing detection tools. Chapter 6 discusses design trade-offs, practical usage guidelines, and current limitations. Finally, Chapter 7 concludes this dissertation.

Chapter 2. Background

2.1 Solidity Execution Model and Debugging Workflow

Solidity smart contracts execute in the Ethereum Virtual Machine (EVM), a state-based execution environment in which program execution is driven by transactions. Unlike conventional applications that can often be executed directly from source code within an integrated development environment (IDE), Solidity contracts typically undergo several stages before their runtime behavior can be inspected. A conventional Solidity debugging workflow therefore involves compilation, deployment, state initialization and transaction execution, and post-execution debugging.

Step 1: Compile. A Solidity source file is first compiled into EVM bytecode and its associated application binary interface (ABI). Figure 2.1 illustrates this step using Remix IDE. The resulting bytecode constitutes the executable representation of the contract, while the ABI provides the interface through which users and other contracts invoke its externally accessible functions.

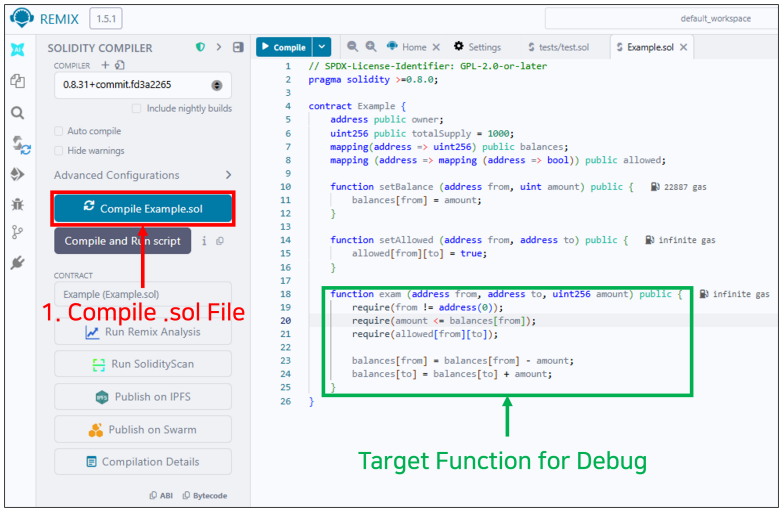


Figure 2.1: Compilation in a conventional Solidity debugging workflow.

Step 2: Deploy. Before a contract can be executed, the compiled bytecode is deployed to a blockchain or a local blockchain environment. As illustrated in Figure 2.2, deployment creates a contract instance and invokes its constructor to initialize the contract state. The deployed contract subsequently maintains persistent state across transactions.

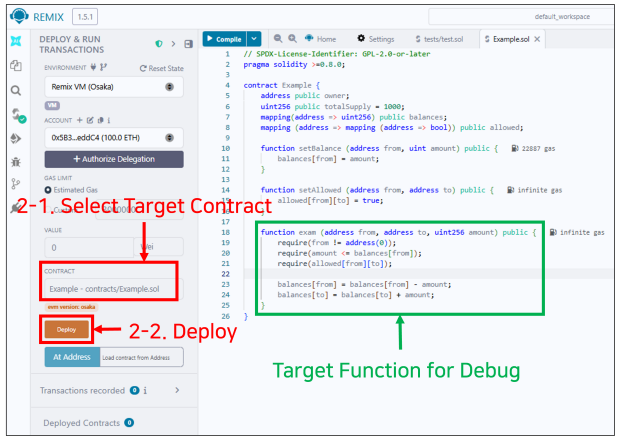


Figure 2.2: Contract deployment in a conventional Solidity debugging workflow.

Step 3: Initialize State and Execute. To inspect a target function, a developer must prepare an execution state in which the function can be invoked. This preparation may

require calling other functions in advance to initialize contract state or satisfy conditions enforced by constructs such as `require`. Once the required state has been established, the developer supplies concrete function arguments and executes the target function as a transaction. Figure 2.3 shows an example of this process in Remix.

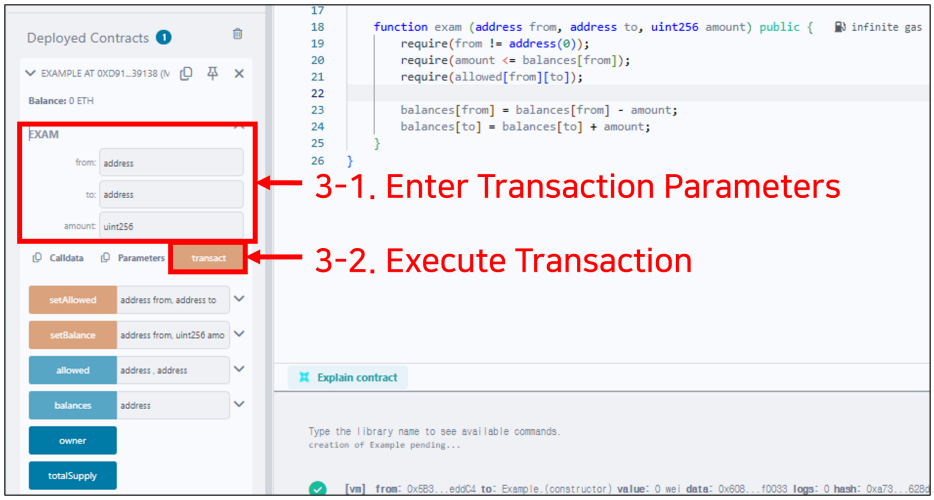


Figure 2.3: State initialization and target-function execution in a conventional Solidity debugging workflow.

Step 4: Debug. After the target transaction has been executed, conventional Solidity debugging tools can expose its execution trace. For example, Remix allows developers to inspect the executed transaction and step through the corresponding EVM instructions, as illustrated in Figure 2.4. The debugger provides information such as the current opcode, stack and memory contents, storage values, and the relationship between bytecode instructions and source-code locations.

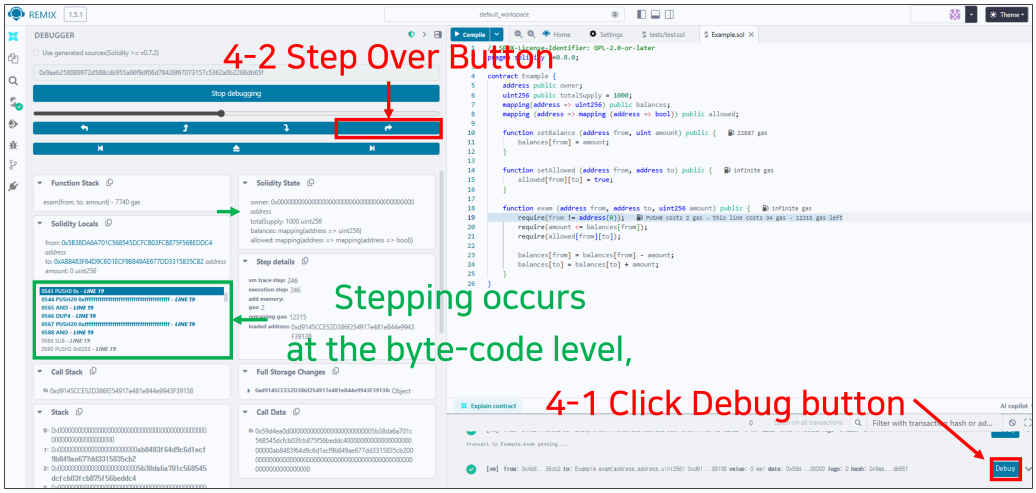


Figure 2.4: Transaction-based bytecode debugging in a conventional Solidity debugging workflow.

This workflow differs fundamentally from the edit–execute–debug cycle commonly available for traditional software. In conventional Solidity debugging, the behavior to be inspected is materialized through a concrete execution: the contract is compiled and deployed, the required state is established, concrete inputs are supplied, and the resulting transaction trace is subsequently examined. Accordingly, the observable debugging context is closely coupled to executable artifacts, initialized contract state, and a particular transaction execution. These characteristics become particularly relevant when debugging support is considered during source-code development, where program behavior may need to be inspected before or while a complete concrete execution context is being prepared.

2.2 Numeric Logic Errors in Solidity

Recent studies have reported a variety of numeric-related weaknesses in smart contracts [4, 25, 28, 35, 37]. Unlike failures that trigger compilation errors, runtime exceptions, or transaction reverts, some numerical defects allow a transaction to complete successfully while producing values that are inconsistent with the intended behavior of the contract. Following prior work, we refer to such defects as *numeric logic errors*.

Definition 2.1 (Numeric Logic Error)**.** A numeric logic error is a bug that allows a transaction to complete successfully without compile-time or runtime exceptions, but violates the developer's intended relationships among numerical values during function execution.

This definition emphasizes an important characteristic of numeric logic errors. Whether a numerical computation is erroneous cannot always be determined solely from the fact that execution succeeds or from the syntactic form of the computation. Instead, correctness depends on whether the resulting numerical relationships conform to the behavior intended by the developer. For example, two arithmetic expressions may both be syntactically valid and execute without reverting, while only one preserves the intended relationship among balances, exchange amounts, rewards, or accounting variables. Thus, the intended numerical relationship constitutes part of the criterion by which such program behavior is judged.

Numeric logic errors can be distinguished according to the execution scope in which they manifest. *Single-transaction errors* arise within one atomic transaction. The relevant computations may span multiple internal or external function calls, but the behavior leading to the erroneous numerical result is contained within a single transaction. In contrast, *multiple-transaction errors* emerge only through a sequence of transactions and therefore depend on state changes accumulated across multiple function invocations. Examples include Risky First Deposit and Price Oracle Manipulation [28, 35], whose effects become observable only after contract state has evolved over multiple transactions.

This dissertation primarily focuses on single-transaction numeric logic errors, including cases in which a transaction crosses contract boundaries through external calls. This scope is consistent with the development-time analysis considered throughout the dissertation: the behavior of interest is bounded by one transaction and can therefore be analyzed with respect to the source-level execution being examined. Numeric properties that inherently span arbitrary transaction sequences require reasoning about inter-transaction dependencies and contract-level invariants and are left outside the primary scope of this work.

Within the single-transaction scope, representative numeric logic errors can be organized according to the abstraction level at which the erroneous behavior arises. Table 2.1

Table 2.1: Representative single-transaction numeric logic error patterns from prior work, classified by abstraction level.

Pattern	Description
Value-level errors: incorrect numerical values arise directly from computations.	
Div In Path [4]	Division results used in conditions may alter control-flow decisions because of integer truncation.
Exchange Problem [4]	Rounding behavior in exchange calculations may produce an unintended output, such as zero output for a valid input or an imbalanced exchange.
Precision Loss Trend [4]	Repeated or improperly ordered integer arithmetic may systematically introduce truncation that favors one side of a computation.
Minor Amount Retention [4]	Integer division in distribution logic may leave numerical remainders unallocated.
Algorithm-level errors: incorrect numerical behavior arises from the structural arrangement of operations.	
Operator Order Issue [4]	Different arithmetic evaluation orders can produce different integer results because intermediate truncation is not preserved across algebraically equivalent expressions.
Erroneous Accounting / Wrong Interest Rate Order [28, 35]	Balance calculations use stale or prematurely updated interest-rate information, resulting in an incorrect numerical outcome.
Erroneous Accounting / Wrong Checkpoint Order [28, 35]	State updates and checkpoints are performed in an incorrect order, causing subsequent reward or accounting calculations to use unintended values.
Inconsistent State Updates [35]	A required state update is omitted, leaving subsequent computations to operate on stale numerical state.
Greedy Contract [25]	The contract accepts funds but lacks the corresponding operation needed to release them, resulting in an unintended numerical state.

summarizes representative patterns reported in prior studies.

Value-level errors arise when the computation directly produces an unintended numerical value. Typical examples include the use of an incorrect operator or identifier, precision loss caused by integer arithmetic, or an arithmetic expression whose result differs from the value intended by the developer. The erroneous behavior is therefore observable primarily through the values produced by the computation.

Algorithm-level errors, in contrast, arise from the structural arrangement of operations. Examples include executing otherwise valid operations in an incorrect order, omitting a required state update, or failing to perform a necessary procedure call. Although

their underlying cause is structural, their effect may still become observable as an unintended relationship among numerical values.

The distinction between value-level and algorithm-level errors provides an organizing basis for characterizing the numeric logic errors considered in this dissertation. The next chapter reviews related work on Solidity debugging and analysis, including existing approaches to numeric logic error detection.

Chapter 3. Related Work

3.1 Solidity Debugging and Development Environments

Modern Solidity development environments either embed a debugger or integrate external debugging plug-ins. Remix IDE [21] is a widely used web-based development environment that provides Solidity editing, compilation, deployment, and transaction debugging. Its debugger operates over executed transactions and allows developers to inspect EVM-level information such as instructions, stack, memory, and storage. Hardhat [13] is a Node.js-based development framework that provides Solidity compilation, local blockchain execution, testing, and deployment support. Visual Studio Code extensions can integrate with Hardhat to provide step-by-step debugging of transaction executions. Forge [11], part of the Foundry toolchain, focuses on fast and reproducible testing. It executes Solidity tests on a local EVM and supports debugging of concrete test executions through Forge Debug. Solidity Debugger Pro [24] is a Visual Studio Code extension that similarly supports runtime debugging over concrete transactions in conjunction with Hardhat.

These environments provide practical support for inspecting concrete smart-contract executions, but their debugging workflows are generally tied to compiled artifacts, deployed or locally instantiated contract state, and executed transactions. In contrast, the approach presented in this dissertation focuses on inspecting program behavior directly at the Solidity source-code level without requiring a separate concrete transaction execution for each debugging scenario.

3.2 Numeric Logic Error Detection in Solidity

Several studies have proposed techniques for detecting numeric and logic-related errors in Solidity smart contracts. These approaches differ in the types of numerical defects they target and in the criteria used to identify erroneous behavior.

NumScout [4] focuses specifically on numerical defects in smart contracts. It defines five defect patterns—Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, and Minor Amount Retention—and detects their occurrences using symbolic execution guided by an LLM-based pruning mechanism. Its detection criteria are therefore derived from predefined numerical defect patterns. GPTScan [28] targets a broader set of logic vulnerabilities, including accounting-related errors. It combines GPT-based scenario matching with lightweight static analysis [10] to identify code that conforms to predefined vulnerability scenarios. Among its supported cases are numerical logic errors involving incorrect accounting operations, update ordering, and related financial computations. ScType [34] approaches numerical correctness from a type-based perspective. It extends Solidity variables with financial type information represented as tuples of $\langle$financial type, scaling factor, token unit, address$\rangle$ and detects inconsistent financial operations through type checking. Unlike pattern-based detectors, ScType can incorporate developer-provided type annotations when the required financial information cannot be inferred directly from the source code.

Other studies address broader forms of logic errors beyond single-transaction numerical behavior. For example, SmartInv [30] infers invariants from source code and natural-language artifacts to detect machine-unauditable bugs, while several approaches analyze multi-transaction behaviors such as price manipulation [1, 16, 29, 32, 33] and flash-loan attacks [17, 20]. These approaches extend logic-error detection to properties that depend on inferred invariants or state changes across multiple transactions.

Overall, existing numeric logic error detectors primarily identify erroneous behavior through predefined defect patterns, vulnerability scenarios, or domain-specific consistency rules. They provide automated detection for the classes of errors represented by their respective analysis models. The approach presented in this dissertation instead focuses on validating numerical relationships explicitly supplied by developers during development.

3.3 Specification-Based Smart Contract Analysis

Specification-based smart contract analysis allows developers to explicitly state expected program properties and check whether contract implementations satisfy them using testing, symbolic execution, or formal verification backends.

Scribble [8] translates developer-written specifications into Solidity assertion code so that the resulting properties can be checked using existing analysis and testing tools such as MythX [7] and Mythril [6]. It supports several forms of specification, including `if_succeed` for conditions on successful function completion, invariant for contract-level properties, `if_updated` for conditions associated with state updates, and `assert` for properties at specific execution points. SmartACE [31] builds on Scribble specifications and verifies Solidity contracts through LLVM-IR-based verification harnesses using tools such as SeaHorn [12], libFuzzer [18], and KLEE [3]. It also abstracts the Ethereum address space into a finite set of representative users to reason about properties involving multiple participants and aggregate accounting behavior. Idealist [19] distinguishes between annotations that provide assumptions for verification, such as loop invariants, call invariants, and call modifies, and annotations that express verification obligations, including `ensures`, `reverts_if`, and contract invariants. These annotations support reasoning about function postconditions, revert behavior, and contract-level invariants. Solvent [2] focuses on user-specified liquidity properties. It translates both the contract and the supplied properties into SMT constraints and checks their satisfiability, producing counterexamples when a property can be violated.

These approaches demonstrate the value of explicit developer-supplied specifications for checking smart-contract behavior. They generally target broad contract properties and rely on verification, symbolic reasoning, or instrumented assertion checking to determine whether those properties hold. In contrast, the approach presented in this dissertation focuses specifically on developer-authored numerical relationships associated with source-level program states and evaluates them over abstract program states during debugging.

3.4 Abstract Interpretation and Interactive Static Analysis

Interactive and incremental abstract interpretation has been actively studied for traditional programming languages. Several studies have integrated incremental or demand-driven analysis into development environments for languages such as C and Java, providing analysis results while programs are being edited [5, 9, 22, 26, 27].

Stein et al. [26] proposed demanded abstract interpretation, which incrementally reconstructs only the analysis components affected by program changes. A subsequent study by Stein et al. [27] extended this approach to procedure summaries, enabling interprocedural analysis results to be reused across edits. Erhard et al. [9] introduced incremental analysis support for Goblint, selectively recomputing affected analysis facts for multithreaded C programs. For Java, Riouak et al. [22] presented IntraJ, an analyzer integrated through the Language Server Protocol that computes only the syntax and data-flow information required for the current development context. Chimdyalwar [5] proposed an efficient interval-analysis technique over call graphs and later extended it with incremental analysis that revisits only functions affected by program modifications.

These studies demonstrate the feasibility of interactive and incremental abstract interpretation for providing responsive analysis during development. In contrast, IQDebug uses program-point abstract states as a source-level debugging context for Solidity and evaluates developer-authored numerical intent over these states.

Chapter 4. The Design of IQDebug

4.1 System Architecture

Figure 4.1 illustrates the overall architecture of IQDebug. The system processes its inputs through three phases: source processing, analysis, and intent validation.

Input. IQDebug receives four types of input:

- **Solidity source fragments** represent statements or code fragments to be incorporated into the currently analyzed program.
- **Debugging inputs** specify values used to establish a debugging scenario.
- **Developer intent** describes numerical relationships to be validated over the analyzed program states.
- **Dependency CFGs** provide precomputed control-flow information for referenced contracts when cross-contract analysis is required.

Phase 1: Source Processing. The Source Processing Module receives the source fragment, debugging inputs, and developer intent. The Solidity compiler checks whether the incoming source fragment can be inserted into the existing contract source without introducing syntactic errors. The ANTLR-based data parser processes the source fragment, debugging inputs, and developer intent according to the extended Solidity grammar. The parsed source and debugging information is forwarded to the Analysis Module, while the parsed intent information is provided to the Intent Validation Engine.

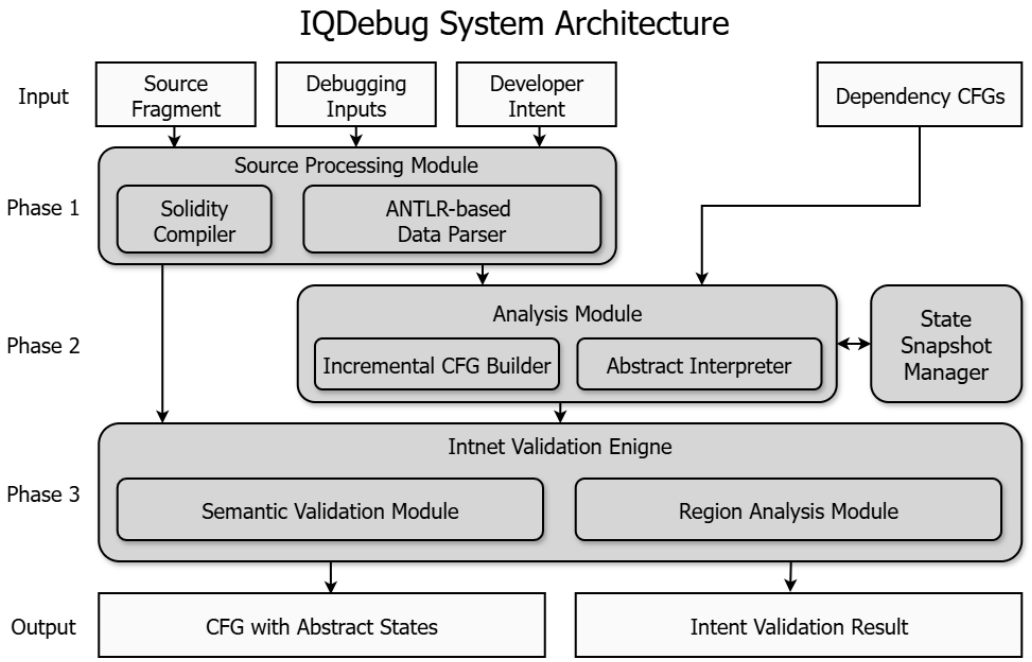


Figure 4.1: Overall architecture of IQDEBUG.

Phase 2: Analysis. The Analysis Module incrementally updates the source-level control-flow graph and computes abstract program states. The Incremental CFG Builder determines where a newly processed source statement should be inserted into the existing CFG, while the Abstract Interpreter evaluates the statement and propagates its effects to affected downstream program points. When external contract behavior is required, precomputed dependency CFGs are supplied to the Analysis Module to support cross-contract analysis. Inherited contract code and referenced libraries are analyzed within the target contract's execution context, while calls to separately deployed external contracts are analyzed only when the corresponding dependency CFG is available. If such a dependency CFG is unavailable, the callee's internal state transitions are not incorporated into the analysis, and values returned from the call may remain unknown unless they are explicitly supplied as debugging inputs. The State Snapshot Manager preserves the analysis state before debugging inputs are applied. After the debugging inputs are interpreted and the corresponding abstract states are computed, the preserved state can be restored so that separate debugging scenarios do not permanently modify the underlying analysis

context. The resulting CFG stores the computed abstract states at its program points and serves as the primary representation used for source-level debugging.

Phase 3: Intent Validation. The Intent Validation Engine evaluates developer intent against the abstract states produced by the Analysis Module. The Semantic Validation Module determines whether the analyzed behavior satisfies, violates, or cannot conclusively establish the specified numerical relationship. The Region Analysis Module further analyzes relevant abstract value regions to provide diagnostic information associated with the validation result.

Output. IQDebug produces two main outputs:

- **CFG with abstract states** is a source-level control-flow graph in which abstract program states are stored at individual program points.
- **Intent validation result** reports the outcome of evaluating developer intent against the corresponding abstract program states.

These outputs are used by the debugging interface to inspect program behavior directly at the Solidity source-code level. Because abstract states are computed in advance and stored at source-level program points in the CFG, the debugger can reuse these stored results as long as the source code and debugging inputs remain unchanged. Under this condition, stepping or moving between breakpoints only requires reading the stored states rather than re-running the Abstract Interpreter at every navigation step.

4.2 Source-Level Program Analysis

IQDebug analyzes Solidity programs directly at the source-code level. The analysis maintains a source-level control-flow graph that evolves as source fragments are added or modified, and computes abstract program states over this representation. This section first describes how Solidity source code is represented and incrementally incorporated into the CFG, then presents the debugging input model and the abstract interpretation used to compute program states.

4.2.1 Source-Level Program Representation

IQDebug maintains a source-level representation of a Solidity program that evolves with source-code edits. Rather than requiring a complete program to be processed at once, IQDebug incrementally incorporates individual source fragments into the existing representation. This process consists of parsing incoming source fragments, representing their control-flow semantics in the source-level CFG, maintaining mappings between source locations and CFG nodes, and updating the representation as the source code is inserted, deleted, or modified.

Source fragments and interactive parsing

IQDebug processes Solidity source code at the granularity of source fragments. The accepted fragments are divided into two types:

- **Block fragment:** Includes contract and function definitions and control-flow constructs such as `if/else` blocks and loops. When the developer types an opening `{`, most editors automatically insert the corresponding closing `}`, allowing the structural block to be processed before its body is fully populated.
- **Statement fragment:** Includes declarations, assignments, function calls, and other expressions that terminate with a semicolon. Each statement can be processed individually when it is completed by the developer.

A source fragment does not necessarily constitute a complete Solidity program by itself. IQDebug therefore uses an ANTLR-based parser with an extended Solidity grammar that provides specialized entry rules for incremental source processing. These rules allow individual fragments to be parsed according to both their syntactic form and their relationship with the source code that has already been processed. Table 4.1 summarizes the seven entry rules for Solidity source constructs.

As shown in Table 4.1, the entry rules are divided into *primary* and *continuation* rules.

- **Primary rules** handle constructs that can be parsed without depending on a preceding source construct.

Table 4.1: Interactive parser entry rules

Type	Entry Rule	Purpose
Primary	interactiveSourceUnit	Top-level declarations: functions, contracts, interfaces, libraries, state variables, pragmas, imports
	interactiveBlockUnit	Statements and control-flow skeletons inside function bodies
Continuation	interactiveEnumUnit	Enum member items added after the enum shell is defined
	interactiveStructUnit	Struct member declarations added after the struct shell is defined
	interactiveDoWhileUnit	The while tail of a do-while loop
	interactiveIfElseUnit	else or else-if branches following an if statement
	interactiveCatchClauseUnit	catch clauses following a try statement

- **Continuation rules** handle fragments whose syntactic role depends on an already existing construct. For example, `interactiveIfElseUnit` requires a preceding `if` or `else if` statement to determine where an incoming `else` branch belongs.

IQDebug also verifies that each source edit preserves the syntactic validity of the complete Solidity program. The incoming fragment is incorporated into the currently maintained source code to construct a candidate program, which is then checked by the Solidity compiler. If the resulting program is syntactically invalid, the edit is rejected before the source-level CFG is updated. Otherwise, the parsed source construct is forwarded to the Incremental CFG Builder, which incorporates its control-flow structure into the existing CFG.

Source-Level CFG Representation

IQDebug represents Solidity programs using a hierarchical source-level CFG. The CFG serves as the common representation for incremental source processing, abstract interpretation, and subsequent source-level debugging. Each node corresponds to a source-level program point, allowing abstract program states and intent-validation results to be

associated with positions in the Solidity source code.

CFG hierarchy. IQDebug organizes the CFG at two levels. At the contract level, a `ContractCFG` represents state-variable initialization, constructor execution, and the functions defined in the contract (Figure A.1). State variables are initialized before the constructor, and the constructor may subsequently update the initialized state. Each function is represented by a separate `FunctionCFG` that models control flow from function entry to its possible termination points (Figure A.2). When a function declares modifiers, the modifier control flow surrounds the function body, with the modifier placeholder (`_`) marking the position at which the function body is inserted. For function calls, the corresponding callee CFG is connected to the caller's control-flow context. If the callee belongs to an external contract whose dependency CFG is available, IQDebug uses the supplied dependency CFG to represent the called function.

CFG node types. Both `ContractCFG` and `FunctionCFG` are composed of source-level CFG nodes that represent different control-flow roles. IQDebug uses the following node types:

- `entry node`: Represents the unique entry point of a contract-level or function-level CFG.
- `state variable node`: Represents contract state-variable declarations. State-variable declarations are grouped into a contract-level node because they do not introduce control-flow branches.
- `basic node`: Represents a source-level statement with sequential control flow, including variable declarations, assignments, function calls, `return`, and `revert` statements.
- `condition node`: Represents constructs that divide control flow according to a condition, including `if`, `else if`, loops, `require`/`assert`, and `try` constructs.
- `join node`: Represents a program point at which control flows from alternative branches are merged.
- `fixpoint evaluation node` (ϕ): Represents the merge point for successive loop iterations and is used for fixpoint computation during abstract interpretation.

- `loop exit node`: Represents the control-flow point reached when execution leaves a loop through its false condition or a corresponding loop-exit path.
- `return exit node`: Represents the common destination of return paths within a function.
- `error exit node`: Represents exceptional termination caused by constructs such as `revert` or failed `require`/`assert` conditions.
- `exit node`: Represents the unique normal exit point of a contract-level or function-level CFG.

Statement-local CFG construction. Within a `FunctionCFG`, each Solidity statement is translated into a CFG structure according to its control-flow semantics. This statement-local representation defines how sequential execution, branching, loops, and abrupt control-flow changes are represented before incremental source updates are applied.

- **Simple statements** create a single `basic node` between the preceding and succeeding control-flow points. This category includes variable declarations, assignments, and function calls (Figure A.3).
- **If statements** create a `condition node`, true and false branches, and an `if join` at which the branch flows are merged (Figure A.4).
- **Else-if clauses** extend the false branch of the preceding conditional with an additional `condition node` and branch structure before reconnecting to the outer join (Figure A.5).
- **Else clauses** replace the remaining false branch with an explicit alternative branch that reconnects to the corresponding join node (Figure A.6).
- **Loops** create a `fixpoint evaluation node` (ϕ), a loop-condition node, a loop-body path, and a `loop exit node`. The body reconnects to the fixpoint node so that repeated iterations can be summarized during abstract interpretation (Figure A.7).

Table 4.2: Line-to-node index mapping by statement type

Statement	Maps Start Line To	Maps End Line To
Simple statements		
`simple statement`	basic node	(none)
`break`	basic node	(none)
`continue`	basic node	(none)
`return`	basic node	(none)
`revert`	basic node	(none)
`require/assert`	condition node	(none)
Compound statements		
`if`	condition node	join node
`else if`	condition node	join node
`else`	basic node	join node
`while`	condition node	loop exit node

- **Break statements** redirect control flow to the corresponding `loop exit node` (Figure A.8), whereas **continue statements** redirect control flow to the loop's ϕ node (Figure A.9).
- **Return statements** redirect control flow to the function's unique `return exit node` (Figure A.10).
- **Require/assert statements** are represented by a `condition node`. The successful branch continues normal execution, while the failing branch leads to the `error exit node` (Figure A.11).
- **Revert statements** terminate the current execution path by connecting directly to the `error exit node` (Figure A.12).

Line-to-Node Mapping

The statement-local CFG structures described above define how individual Solidity constructs are represented in the control-flow graph. For incremental source processing,

however, IQDebug must additionally maintain a correspondence between source-code locations and the CFG nodes representing those locations. This correspondence is required because a newly inserted source fragment is identified by its position in the source code, whereas the CFG itself represents control-flow relationships rather than textual positions. IQDebug therefore maintains a line-to-node index M that associates source lines with the corresponding CFG nodes. The mapping reflects the CFG structure created for each statement type, as summarized in Table 4.2.

Simple statements require only a mapping from their source line to the node that directly represents the statement. Most simple statements therefore map to a `basic node`, whereas `require` and `assert` map to a `condition node` because their successful and failing outcomes form separate control-flow paths.

Compound constructs additionally require their structural boundaries to be represented. For `if` and `else if` statements, the start line maps to the corresponding `condition node`, while the end line maps to the `join node` at which their branch flows converge. An `else` block maps its start line to the entry of the alternative branch and its end line to the corresponding join node. Similarly, a loop maps its start line to the loop condition and its end line to the `loop exit node`.

The line-to-node mapping provides the bridge between textual source edits and their corresponding positions in the CFG. IQDebug uses this mapping to locate insertion points, identify deletion boundaries, and determine the nodes affected by source modifications.

Incremental Source-Code Updates

IQDebug supports three types of incremental source-code updates: insertion, deletion, and modification. Each update modifies the existing source-level CFG in place rather than reconstructing the entire graph. After an update, abstract interpretation is resumed from the affected program point so that changes are propagated to downstream states while unaffected analysis results can be reused.

Code insertion. When a new source fragment is inserted, IQDebug first determines the CFG position at which the corresponding control-flow structure should be attached. The lookup procedure differs depending on whether the incoming fragment requires a previ-

Algorithm 1 Dependent-Context Lookup

Input: CFG G, line-to-node index M, current line L
Output: Condition node c
function GETBRANCHCONTEXT(G, M, L)
    $queue \leftarrow$ `FindJoinNode`(M, L) ▷ find join node at or before line L
    if $queue = \emptyset$ **then**
        error "No join node found at or before line L"
    end if
    $visited \leftarrow \emptyset$
    while $queue \neq \emptyset$ **do**
        $n \leftarrow$ `Dequeue`$(queue)$
        if $n \in visited$ **then**
            continue
        end if
        $visited \leftarrow visited \cup \{n\}$
        if `IsCond`(G, n) **and** `CondType`$(n) \in \{$`if`, `else_if`$\}$ **then**
            return n
        end if
        for $p \in$ `Pred`(G, n) **do**
            if $p \notin visited$ **then**
                `Enqueue`$(queue, p)$
            end if
        end for
    end while
    error "No matching condition node found"
end function

ously established control-flow context.

- **Dependent context:** A continuation construct such as `else` or `else if` cannot be inserted independently. IQDebug uses the line-to-node index together with CFG predecessor relationships to locate the preceding conditional to which the construct belongs.
- **Independent context:** Other statements and blocks can be positioned using the neighboring source-to-node mappings. IQDebug identifies the CFG node following the insertion position and derives the corresponding predecessor from the surrounding CFG structure.

Algorithms 1 and 2 describe these two cases.

Dependent-context lookup. Algorithm 1 locates the control-flow context required for continuation constructs such as `else` and `else if`, which cannot be positioned independently.

- **Initialization (Lines 4–8):** The algorithm uses the line-to-node index M to find the join node at or before the current source line L. This node provides a starting point for backward traversal because a continuation construct must be associated with a previously established conditional structure. If no corresponding join node exists, the requested continuation cannot be resolved in the current CFG.

- **Predecessor traversal (Lines 9–22):** Starting from the identified join node, the algorithm performs a breadth-first search over CFG predecessors. The traversal continues until it finds the nearest `condition node` corresponding to an `if` or `else if` construct. This node represents the conditional context to which the incoming continuation fragment should be attached.

- **Failure handling (Line 23):** If the traversal terminates without locating a matching conditional node, the algorithm reports that no valid dependent context exists for the incoming fragment.

Independent-context lookup. Algorithm 2 determines the CFG position for source fragments that do not require a preceding continuation context. The algorithm follows a successor-oriented strategy, using the first mapped node after the insertion position as the primary structural reference.

- **Successor lookup (Line 4):** The algorithm first retrieves the first CFG node mapped after source line L using `FindPostNode`. Because this succeeding source construct has already been incorporated into the CFG, it provides a stable reference for determining where the new fragment should be inserted.

- **Structural-boundary case (Lines 5–11):** If the succeeding node is a `loop exit node` or `join node`, the insertion position lies immediately before the boundary of a loop or conditional construct. The algorithm therefore retrieves the preceding

Algorithm 2 Independent-Context Lookup

```
1:  Input: CFG G, line-to-node index M, insertion position at line L
2:  Output: Insertion-site node A
3:  function GETINSERTIONSITE(G, M, L)
4:      s ← FindPostNode(M, L)                          ▷ find first node after line L
5:      if IsLoopExit(G, s) or IsJoin(G, s) then
6:          n ← FindPreviousNode(M, L)
7:          if IsCond(G, n) then
8:              return BranchBlock(n, true)
9:          else
10:             return n
11:         end if
12:     else
13:         pred ← Pred(G, s)
14:         if |pred| = 1 then
15:             return the unique element of pred
16:         else
17:             error "Basic successor must have exactly one predecessor"
18:         end if
19:     end if
20: end function
```

mapped node using `FindPreviousNode`. If this node is a condition, its true branch is returned as the insertion context; otherwise, the preceding node itself becomes the insertion point.

- **Sequential-successor case (Lines 12–19):** If the succeeding node is an ordinary sequential node, the algorithm retrieves its predecessor. A sequential node has a unique predecessor by construction, and this predecessor is returned as the insertion point. If this invariant does not hold, the CFG is considered structurally inconsistent.

Together, Algorithms 1 and 2 translate a source-code insertion position into the corresponding CFG context. Once the context is identified, the Incremental CFG Builder creates the statement-specific CFG structure described in Section 4.2.1, updates the surrounding edges and line-to-node mappings, and resumes incremental interpretation from the affected program point.

Algorithm 3 Code Deletion

1: **Input:** Function-level CFG G, line-to-node index M, source code C, deletion range $[L_s, L_e]$
2: **Output:** Updated CFG with reconnected edges and recomputed abstract states
3: **function** CODEDELETION(G, M, C, L_s, L_e)
4: $C' \leftarrow$ `RemoveLines`(C, L_s, L_e) ▷ produce candidate source
5: **if** $\neg$ `SyntaxValid`(C') **then**
6: **error** "Deletion produces invalid syntax"
7: **end if**
8: $pred, succ \leftarrow$ `FindBoundary`(G, M, L_s, L_e) ▷ unique pred/succ outside range
9: G.`RemoveEdge`$(pred,$ `FindNode`$(M, L_s))$ ▷ detach deleted region
10: G.`RemoveEdge`(`FindNode`$(M, L_e), succ)$
11: G.`AddEdge`$(pred, succ)$ ▷ reconnect
12: **for all** $l \in [L_s, L_e]$ **do**
13: G.`RemoveNodes`$(M[l])$ ▷ remove all nodes and their internal edges
14: **end for**
15: `ShiftIndex`$(M, L_e, L_e - L_s + 1)$ ▷ shift subsequent line mappings
16: `IncrementalInterpretation`$(G, \{succ\})$ ▷ Algorithm 4
17: **end function**

Code deletion. Algorithm 3 removes a selected source range while preserving the consistency of both the Solidity source and the corresponding function-level CFG.

- **Syntax validation (Lines 4–7):** The algorithm first removes the source lines in the deletion range $[L_s, L_e]$ from C to construct a candidate source C'. The candidate source is then checked by the Solidity compiler. If the deletion makes the program syntactically invalid, the update is rejected before the CFG is modified.

- **Boundary lookup (Line 8):** Using the line-to-node index M, `FindBoundary` identifies the CFG region corresponding to the deleted source range and returns the predecessor *pred* and successor *succ* immediately outside that region. These nodes define the control-flow boundaries that must be reconnected after deletion.

- **CFG reconnection (Lines 9–11):** The edges connecting the deleted region to *pred* and *succ* are removed, and a new edge is created directly from *pred* to *succ*. This bypasses the deleted control-flow region while preserving the surrounding CFG structure.

- **Node removal (Lines 12–14):** All CFG nodes associated with source lines in $[L_s, L_e]$ are removed together with their internal edges.
- **Index update and reinterpretation (Lines 15–16):** Because source lines following the deleted range shift upward, IQDebug updates the corresponding entries in the line-to-node index. Incremental interpretation then resumes from *succ*, propagating the effects of the deletion only to downstream program points affected by the structural change.

Code modification. Code modification handles edits that change the contents of an existing statement without changing its control-flow role, such as modifying an assigned expression or changing the condition of an `if` statement. IQDebug uses the line-to-node mapping to locate the CFG node corresponding to the modified source statement and replaces the stored statement while preserving the surrounding CFG structure. Incremental interpretation is then resumed from the modified node so that the effects of the change are propagated to downstream abstract states.

4.2.2 Debugging Input Model

IQDebug allows developers to establish a debugging scenario by directly specifying values for program variables relevant to the target analysis. Unlike transaction-based state preparation, these debugging inputs are injected into the abstract analysis before the target program behavior is evaluated.

IQDebug supports four types of debugging input:

- `@GlobalVar`**:** Specifies a value for a Solidity global variable or a value derived from the global execution context, such as `msg.sender`, `block.timestamp`, or an address balance.
- `@StateVar`**:** Specifies a value for a contract state variable. The referenced variable may include nested member or index accesses, allowing values to be supplied for mappings and structured state variables.
- `@Param`**:** Specifies a value for a parameter of the analyzed function at function entry.

```
debugInput
: debugGlobalVar
| debugStateVar
| debugParam
| debugIReturn
;

debugGlobalVar
: '@GlobalVar' identifier ('.' identifier)? '=' debugValue
| '@GlobalVar' 'address' '(' identifier ')' '.'
identifier '=' debugValue
;

debugStateVar
: '@StateVar' varRef '=' debugValue
;

debugParam
: '@Param' varRef '=' debugValue
;

debugIReturn
: '@IReturn' identifier '.' identifier '(' ')'
ireturnAccessChain? '=' debugValue
| '@IReturn' identifier '(' identifier ')' '.'
identifier '(' ')' ireturnAccessChain? '=' debugValue
;

debugValue
: '[' signedNumberLiteral ',' signedNumberLiteral ']'
| 'symbolicAddress' numberLiteral
| 'symbolicBytes' hexStringLiteral
| 'symbolicString' hexStringLiteral
| ('true' | 'false' | 'any')
| identifier ('.' identifier)*
| 'array' '['(signedNumberLiteral (',' signedNumberLiteral))?']'
| 'arrayAddress' '['(numberLiteral (',' numberLiteral))?']'
;

// Other grammar rules are omitted for brevity.
```

Figure 4.2: Grammar of the debugging input.

- `@IReturn`**:** Specifies a return value for an external call made through a Solidity interface. An interface declares the callable function signatures but does not contain the implementation of the external contract that will execute the call. There-

fore, when the corresponding dependency CFG is unavailable, IQDebug cannot derive the returned value from the available source-level representation. Similar to `@GlobalVar`, `@StateVar`, and `@Param`, `@IReturn` allows the developer to provide this value as part of a debugging scenario and inspect the subsequent source-level behavior under the specified input. `@IReturn` is restricted to interface functions declared `view` or `pure`. For state-modifying external calls, supplying only a return value would not represent the state changes that may affect subsequent program behavior.

Figure 4.2 presents the grammar of these debugging inputs. The grammar distinguishes the target of a debugging input from the value assigned to that target. `@GlobalVar` supports Solidity global variables and address-related global properties, while `@StateVar` and `@Param` use `varRef` to reference state variables and function parameters, respectively. `@IReturn` identifies an external function call and may additionally include member, index, or chained-call accesses over the returned value.

The `debugValue` rule defines the values that can be injected into the analysis. Numerical values are specified as intervals using two signed bounds, while non-numerical inputs are represented using symbolic or finite abstract forms. Specifically, IQDebug supports symbolic addresses, symbolic byte and string values, Boolean values including `any`, enum literals, integer arrays, and address arrays. These forms allow debugging inputs to describe both numerical ranges and non-numerical execution contexts without requiring a concrete transaction state.

4.2.3 Abstract Interpretation of Program States

IQDebug uses abstract interpretation to compute abstract values of program variables at source-level program points. For numerical variables, the analysis uses the interval domain, where an interval such as $[1, 10]$ represents all integer values from 1 through 10 as a single abstract value. Non-numerical Solidity values are represented using the corresponding abstract domains. The complete domain definitions and formal semantics are provided in Appendix B. At each CFG program point, IQDebug maintains an abstract environment that maps program variables to abstract values. An unconstrained value may be represented by $\top$, whereas an infeasible program state is represented by $\bot$. When

multiple control-flow paths reach the same program point, their abstract environments are combined using the least upper bound operator $\sqcup$. For conditional branches, the incoming environment is refined according to the branch predicate before it is propagated to the corresponding successor.

Abstract interpretation is performed even when no debugging input is provided. In this case, values that can be derived from the source code, such as literals, initialized variables, and values constrained by branch conditions, are propagated through the CFG, while values without sufficient constraints may remain $\top$ or otherwise imprecise. When the developer provides debugging inputs, IQDebug reuses the existing CFG and injects the specified values into the analysis environment. The source-level CFG itself is not reconstructed because the program structure remains unchanged. Instead, the target function is reinterpreted using the updated environment, and the abstract states stored at downstream program points are recomputed to reflect the supplied inputs. IQDebug distinguishes between source-code updates and debugging-input updates when determining where reinterpretation begins. For a source-code update, the CFG is first modified as described in Section 4.2.1, and the affected program point is used as the seed of incremental reinterpretation. Only forward-reachable states that may be influenced by the edit need to be recomputed. For a debugging-input update, interpretation restarts from the entry of the analyzed function because the modified input environment may affect all subsequent program points.

Algorithm 4 performs worklist-based propagation from the selected seed nodes. For each node, the incoming environment is constructed from the abstract states of its predecessors. If a predecessor is a condition node, the corresponding branch predicate is used to refine the predecessor state before propagation. At merge points, incoming states from multiple predecessors are combined using $\sqcup$.

The resulting environment is then processed by the abstract transfer function associated with the current node. If the newly computed output differs from the previously stored abstract state, IQDebug updates the node and places its successors on the worklist. If the output is unchanged, propagation from that node stops because the current update introduces no further change to downstream states. This change-driven procedure allows previously computed abstract states to be reused when they are unaffected by a source-code edit.

Algorithm 4 Incremental Interpretation

Input: CFG G, seed set S
Output: Environments updated along forward-reachable paths from S
function INCREMENTALINTERPRETATION(G, S)
    $wl \leftarrow \langle\rangle$; $inq \leftarrow \emptyset$; $out \leftarrow$ output map
    for all $s \in S$ **do**
        if $\neg\texttt{IsSink}(G,s) \wedge s \notin inq$ **then**
            $wl.\texttt{enqueue}(s)$; $inq \leftarrow inq \cup \{s\}$
        end if
    end for
    while $wl \neq \langle\rangle$ **do**
        $n \leftarrow wl.\texttt{dequeue}()$; $inq \leftarrow inq \setminus \{n\}$
        $\hat{\sigma}_{\text{in}} \leftarrow \bigsqcup_{p \in \texttt{Pred}(G,n)} \texttt{PruneByCondition}(p,n)$
        if $\texttt{IsLoopHeader}(G,n)$ **then**
            $exit \leftarrow \texttt{Fixpoint}(G,n)$ ▷ compute loop fixpoint (Algorithm 5)
            for all $s \in \texttt{Succ}(G,exit)$ **do**
                if $\neg\texttt{IsSink}(G,s) \wedge s \notin inq$ **then**
                    $wl.\texttt{enqueue}(s)$; $inq \leftarrow inq \cup \{s\}$
                end if
            end for
            continue
        end if
        $\hat{\sigma}_{\text{out}} \leftarrow \texttt{Transfer}(n, \hat{\sigma}_{\text{in}})$
        if $\hat{\sigma}_{\text{out}} \neq out[n]$ **then**
            $Env(n) \leftarrow \hat{\sigma}_{\text{out}}$; $out[n] \leftarrow \hat{\sigma}_{\text{out}}$
            for all $s \in \texttt{Succ}(G,n)$ **do**
                if $\neg\texttt{IsSink}(G,s) \wedge s \notin inq$ **then**
                    $wl.\texttt{enqueue}(s)$; $inq \leftarrow inq \cup \{s\}$
                end if
            end for
        end if
    end while
end function

Loops require separate treatment because their control-flow cycles may otherwise cause abstract states to be propagated indefinitely. IQDebug therefore computes a fixpoint that summarizes the effects of repeated loop execution. With interval analysis, widening is used to ensure convergence, but applying widening too early may expand an otherwise

bounded interval excessively.

To mitigate this loss of precision, IQDebug uses adaptive widening. When the operands of a loop condition can be evaluated to sufficiently precise intervals, including through developer-provided debugging inputs, the analyzer estimates an iteration threshold and delays widening until that threshold is exceeded. For example, when a loop counter and its bound are represented by singleton intervals, their difference can be used to estimate the number of iterations. If such an estimate cannot be derived, the analysis falls back to a default widening threshold.

Algorithm 5 computes the abstract state entering the loop from predecessors outside the loop and repeatedly propagates states through the loop body. Before the widening threshold is reached, successive states are combined using $\sqcup$. Once the threshold is exceeded at the fixpoint evaluation node, widening is applied to ensure convergence. After the widening phase stabilizes, narrowing is applied to refine the resulting overapproximation. The final loop-exit state is then obtained by refining the stabilized state with the negated loop condition and propagating it to the program point following the loop.

The abstract environment computed at each source-level program point is stored in the corresponding CFG node. These stored states are subsequently consumed by both the Intent Validation Engine and the source-level debugging interface. As long as the source code and debugging inputs remain unchanged, debugging navigation can reuse the stored states without re-running abstract interpretation at every step. When the source code or debugging inputs change, the relevant abstract states are recomputed according to the reinterpretation rules described above.

For loop-related program points, the stored abstract state represents the stabilized fixpoint result rather than the state of a particular concrete iteration. This design allows IQDebug to present a finite source-level debugging representation even when the corresponding concrete execution may traverse a loop multiple times.

Algorithm 5 Loop Fixpoint with Adaptive Widening

Input: CFG G, loop condition node h
Output: Loop exit node with stabilized environment
function LOOPFIXPOINT(G, h)
    $L \leftarrow \texttt{LoopNodes}(G,h)$; $start \leftarrow \bigsqcup\{Env(p) \mid p \in \texttt{Pred}(G,h) \setminus L\}$
    $\tau \leftarrow \texttt{EstimateIterations}(h, start)$ ▷ annotation-aware threshold
    $vis[\cdot] \leftarrow 0$; $in[h] \leftarrow start$
    // Widening phase
    $wl \leftarrow \langle h \rangle$
    while $wl \neq \langle\rangle$ **do**
        $n \leftarrow wl.\texttt{dequeue}()$; $vis[n] \leftarrow vis[n] + 1$
        if $in[n] = \bot$ **then**
            $in[n] \leftarrow \bigsqcup_{p \in \texttt{Pred}(G,n) \cap L} \texttt{PruneByCondition}(p, n)$
        end if
        $\hat{o}_{\text{raw}} \leftarrow \texttt{Transfer}(n, in[n])$ ▷ compute raw output
        if $\texttt{IsFixpointEvalNode}(n) \wedge vis[n] > \tau$ **then**
            $\hat{o} \leftarrow \texttt{Widen}(out[n], \hat{o}_{\text{raw}})$ ▷ widen output after τ visits
        else
            $\hat{o} \leftarrow out[n] \sqcup \hat{o}_{\text{raw}}$
        end if
        if $\hat{o} = out[n]$ **then continue** ▷ output unchanged, skip propagation
        end if
        $out[n] \leftarrow \hat{o}$
        for all $s \in \texttt{Succ}(G,n) \cap L$ **do**
            $in[s] \leftarrow in[s] \sqcup \texttt{PruneByCondition}(n, s)$
            $wl.\texttt{enqueue}(s)$
        end for
    end while
    $\texttt{NarrowingPhase}(L)$ ▷ standard descending iteration
    $exit \leftarrow \texttt{FindLoopExit}(G,h)$
    $Env(exit) \leftarrow \bigsqcup_{p \in \texttt{Pred}(G,exit)} \texttt{PruneByCondition}(p, exit)$ ▷ join with loop-exit pruning
    return *exit*
end function

4.3 Intent-Aware Validation

The denotational semantics of the intent model is built around a meaning function

$$\mathscr{V}[\![\cdot]\!] : \mathbb{A} \to \big(\mathbb{C} \rightharpoonup \mathbb{J}\big),$$

which assigns to each intent $a \in \mathbb{A}$ a partial function that maps validation contexts in $\mathbb{C}$ to judgments in $\mathbb{J}$.

Syntactic domain. $\mathbb{A}$ is the set of developer-authored intents generated by the grammar in §4.3, with a typical intent denoted by a. Each intent begins with either `@During` or `@Post` and contains one atomic intent clause.

The grammar distinguishes three classes of forms:

- `@During`**-specific forms**, associated with an individual program statement and evaluated at the corresponding program point;
- `@Post` **forms**, evaluated with respect to the normal function-exit context; and
- **Common forms**, which may be used under either `@During` or `@Post`, with their reference environment determined by the intent scope.

Each intent expresses one atomic numeric obligation. When multiple independently checkable obligations are required, they are represented as separate intents rather than being combined through intent-level logical connectives.

Semantic domain. We use two semantic domains:

- Σ — the space of abstract variable environments produced by the abstract interpreter. Each $\sigma \in \Sigma$ maps program variables to abstract values, including compound values such as arrays, mappings, and structs whose numeric leaves are represented by intervals.
- $\mathbb{C}$ — the set of validation contexts. Each $\Gamma \in \mathbb{C}$ is a record containing the program-state snapshots and auxiliary data available when an intent is evaluated.

Table 4.3 presents the abstract-state snapshots and auxiliary information that constitute the validation context used by IQDebug.

Field supply. The fields available in Γ depend on the intent scope:

- `@During`: σ_{pt}, σ_{before}, σ_{assign}, σ_{entry}, R, and $\mathscr{K}$;
- `@Post`: σ_{pt}, σ_{entry}, σ_{exit}, and R.

Table 4.3: Fields of the validation context $\Gamma \in \mathbb{C}$. Each intent form reads only the fields required by its semantics.

Field	Meaning
$\sigma_{\text{pt}} \in \Sigma$	abstract state at the target program point
$\sigma_{\text{before}} \in \Sigma$	state immediately before the statement at the selected source line
$\sigma_{\text{assign}} \in \Sigma$	state at the relevant variable-assignment point within the enclosing function
$\sigma_{\text{entry}} \in \Sigma$	state at function entry, after parameter binding and supplied debugging inputs
$\sigma_{\text{exit}} \in \Sigma$	joined state at normal function exit
$R : \text{Line} \rightharpoonup \text{Val}^*$	partial map from return-statement lines to captured return values
$\mathscr{K}$	statements associated with the selected source location, used for call-argument and `require`/`assert` lookup

This distinction also determines the available snapshot qualifiers. Under `@During`, `before`, `after`, and `assign` refer to σ_{before}, σ_{pt}, and σ_{assign}, respectively. Under `@Post`, `entry` and `exit` refer to σ_{entry} and σ_{exit}, respectively. The target program-point state σ_{pt} corresponds to the state after the selected statement under `@During` and to the normal function-exit state under `@Post`. Accordingly, an unqualified reference is evaluated using σ_{pt} in both scopes and is equivalent to `after` under `@During` and `exit` under `@Post`.

Judgment set. $\mathbb{J} = S, V, W, U$ is the set of four validation judgments:

- **Satisfied** (S) — every concrete state represented by the relevant abstract state satisfies the intent;
- **Violated** (V) — every represented concrete state violates the intent;
- **Warning** (W) — the target program point is reachable, but the available abstract information is not precise enough to establish either uniform satisfaction or uniform violation;
- **Unreachable** (U) — the target program point has no reachable abstract state under the current analysis context, and therefore the intent cannot be evaluated over any reachable program state.

For an intent $a \in \mathbb{A}$ and a validation context $\Gamma \in \mathbb{C}$, IQDebug reports $\mathscr{V}[\![a]\!](\Gamma) \in \mathbb{J}$. *Satisfied* and *Violated* represent definite validation outcomes. They indicate that the

intent uniformly holds or uniformly fails, respectively, over all concrete states represented by the relevant abstract state. *Warning* instead indicates that the target program point is reachable, but the abstraction contains insufficient information to establish either of these definite outcomes. *Unreachable* is distinct from *Warning*: it indicates that no reachable abstract state exists at the target program point under the current analysis context.

Syntax and Semantics

Grammar overview. Figure 4.3 presents the syntax of developer-authored intents supported by IQDebug.

An intent begins with either `@During` or `@Post`, which determines its evaluation scope and the intent forms and snapshot-qualified references available within the expression. An `@During` intent specifies a condition evaluated at a selected statement-level program point. Its clauses include function-argument relations, feasibility checks for `require` and `assert` statements, and the common forms shared with `@Post`. An `@Post` intent specifies a condition evaluated at the normal exit of the target function and is expressed using the common forms. The common forms support direct relational comparisons, return-related comparisons, percentage- and rounding-based comparisons through `PercentOf`, `ceil`, and `floor`, truthiness-based implication, and state-change checks through `changed`. The return-related forms distinguish three ways of referring to function results. `returnExpression` refers to the complete value captured at a return statement, while `return[`n`]` selects the n-th element of a tuple return. The `return` form instead compares numeric expressions evaluated in the abstract environment associated with the selected return statement.

Intent-value expressions define the numeric operands used by these intent forms. They support numeric literals, inline intervals $[a,b]$, variable references with member and index access, parenthesized expressions, and arithmetic combinations of these values. The supported arithmetic operators include addition, subtraction, multiplication, division, modulo, exponentiation, and bit shifting. The `PercentOf` helper expresses a percentage of an intent-value expression, whereas the second argument of `ceil` and `floor` specifies the unit to which the evaluated value is rounded upward or downward, respectively. The grammar distinguishes two intent-value expression contexts, denoted by $k \in \{D, P\}$,

duringIntent → `@During` duringClause

postIntent → `@Post` postClause

duringClause → identifier `.arg[`n`]` relOp intentValue$_D$ (D_{arg})
| `require` feasible (D_{req})
| `assert` feasible (D_{asrt})
| commonClause$_D$ (D_{com})

postClause → commonClause$_P$ (P_{com})

commonClause$_k$ → returnExpression relOp intentValue$_k$ (C_{ret})
| `return[`n`]` relOp intentValue$_k$ (C_{ret_n})
| `return` intentValue$_k$ relOp intentValue$_k$ (C_{retVar})
| intentValue$_k$ relOp `PercentOf(`intentValue$_k$ `,` p`)` (C_{pct})
| intentValue$_k$ relOp `ceil(`intentValue$_k$ `,` d`)` (C_{ceil})
| intentValue$_k$ relOp `floor(`intentValue$_k$ `,` d`)` (C_{flr})
| intentValue$_k$ relOp intentValue$_k$ (C_{cmp})
| intentValue$_k$ => intentValue$_k$ (C_{impl})
| `changed(`intentValue$_k$ `,` `true` | `false``) (C_{chg})

intentValue$_k$ → arithExpr$_k$

arithExpr$_k$ → arithExpr$_k$ ($\ll$ | $\gg$ | $\ggg$) arithAdd$_k$
| arithAdd$_k$

arithAdd$_k$ → arithAdd$_k$ (+ | -) arithTerm$_k$
| arithTerm$_k$

arithTerm$_k$ → arithTerm$_k$ (* | / | %) arithExp$_k$
| arithExp$_k$

arithExp$_k$ → arithFactor$_k$ ** arithExp$_k$
| arithFactor$_k$

arithFactor$_D$ → number | `[` number `,` number `]` | varRef
| varRef`(before)` | varRef`(after)`
| varRef`(assign)` | `(` arithExpr$_D$ `)`

arithFactor$_P$ → number | `[` number `,` number `]` | varRef
| varRef`(entry)` | varRef`(exit)`
| `(` arithExpr$_P$ `)`

varRef → identifier subAccess*

subAccess → `.` identifier | `[` expr `]`

relOp → $<$ | $>$ | $\leq$ | $\geq$ | `==` | `!=`
| `in` | `not in`

Figure 4.3: Grammar of developer-authored intent.

according to the evaluation scope of the intent. The D context is used for `@During` intents and permits the snapshot qualifiers `before`, `after`, and `assign`. As defined by the validation context in Table 4.3, these qualifiers refer to the abstract states immediately before the selected statement, at the target program point after the statement has been interpreted, and at the relevant assignment of the referenced variable, respectively. The P context is used for `@Post` intents and permits `entry` and `exit`, which refer to the abstract states at function entry and normal function exit.

Unqualified variable references remain valid in both contexts and are evaluated at the target program point. Consequently, an unqualified reference corresponds to `after` under `@During` and to `exit` under `@Post`. Because a snapshot qualifier applies to an individual variable reference, references from different snapshots may be combined within the same arithmetic expression. The supported relational operators are $<$, $>$, $\leq$, $\geq$, `==`, and `!=`, together with the membership operators `in` and `not in`. The implication operator $=>$ is a separate truthiness-based intent form rather than a relational operator. The denotational semantics of these intent forms and operators are defined below.

Well-formedness. We write $\Gamma \vdash a : \textbf{wf}$ to mean that an intent a is well-formed under validation context Γ. An intent is well-formed if it conforms to the grammar in Figure 4.3, every variable reference resolves in the corresponding field of Γ, and all auxiliary information required by the selected intent form is available. The semantic equations below define $\mathscr{V}[\![a]\!](\Gamma)$ only for well-formed inputs. Accordingly, an intent for which a required snapshot or auxiliary field is unavailable lies outside the domain of the meaning function rather than producing a validation judgment.

Well-formedness is independent of the precision of the abstract values contained in the validation context. IQDebug may perform intent validation without any developer-supplied debugging inputs, in which case values that cannot be sufficiently constrained by the program analysis may remain $\top$ or otherwise imprecise. Developers may optionally provide debugging inputs such as `@GlobalVar`, `@StateVar`, `@Param`, and `@IReturn` to constrain such values and obtain more precise abstract states. Even when an intent is well-formed, insufficiently precise abstract values may lead to a *Warning* judgment. By contrast, if the target program point itself has no reachable abstract state, the well-formed intent yields *Unreachable*.

Expression evaluation. Let $\mathscr{I}$ denote the set of integer intervals. Intent-value expressions are evaluated against an abstract variable environment $\sigma \in \Sigma$ by

$$\text{eval} \;:\; \Sigma \times \mathit{intentValue} \;\to\; \mathscr{I}.$$

Ordinary arithmetic subexpressions are evaluated using the underlying abstract interpreter. An inline interval literal a,b is interpreted directly as an abstract integer interval, while a numeric literal n evaluates to the singleton interval n,n.

Snapshot qualification changes the abstract environment used to evaluate an individual variable reference. For a variable reference v,

$$\text{eval}(\sigma, v(\mathit{entry})) = \text{eval}(\sigma_{\text{entry}}, v), \qquad \text{eval}(\sigma, v(\mathit{exit})) = \text{eval}(\sigma_{\text{exit}}, v),$$

$$\text{eval}(\sigma, v(\mathit{before})) = \text{eval}(\sigma_{\text{before}}, v), \qquad \text{eval}(\sigma, v(\mathit{after})) = \text{eval}(\sigma_{\text{pt}}, v),$$

$$\text{eval}(\sigma, v(\mathit{assign})) = \text{eval}(\sigma_{\text{assign}}, v).$$

Unqualified variable references are evaluated using the environment supplied as the first argument of eval. For the intent semantics defined below, this environment is σ_{pt} unless a particular intent form specifies another evaluation environment. Accordingly, an unqualified reference corresponds to `after` under `@During` and to `exit` under `@Post`.

Some intent forms obtain their operands through auxiliary information rather than directly from an *intentValue*. For rule (D_{arg}), $\text{arg}_n(f, \mathscr{K})$ resolves the n-th argument expression of the selected function call, after which the expression is evaluated using eval. Rules (C_{ret}) and (C_{ret_n}) instead obtain captured return values directly from the return-summary field R as $R(\ell_{\text{ret}})$ and $R(\ell_{\text{ret}})[n]$, respectively.

Boolean-expression evaluation. The feasibility forms operate on Solidity Boolean conditions rather than on *intentValue* expressions. We therefore use the abstract Boolean domain

$$\mathscr{B} = \big\{\{0\}, \{1\}, \{0,1\}\big\}$$

and the boolean-expression evaluation function

$$\text{eval}_{\mathscr{B}} \;:\; \Sigma \times \text{BExpr} \;\to\; \mathscr{B},$$

where BExpr denotes the supported Solidity Boolean-expression category. The values 0, 1, and $0,1$ represent conditions that are definitely false, definitely true, and potentially either false or true, respectively.

Numeric helper operations. The intent forms `PercentOf`, `ceil`, and `floor` transform an evaluated integer interval into another integer interval before relational comparison.

For an interval $X = [l,u]$ and an integer percentage p, we define

$$\text{PercentOf}^*(X,p) = \text{hull}\left(\left\lfloor \frac{lp}{100} \right\rfloor, \left\lceil \frac{lp}{100} \right\rceil, \left\lfloor \frac{up}{100} \right\rfloor, \left\lceil \frac{up}{100} \right\rceil\right),$$

where hull denotes the smallest integer interval containing all of its arguments.

For a positive integer unit n, let

$$\text{ceil}_n(x) = n\left\lceil \frac{x}{n} \right\rceil, \qquad \text{floor}_n(x) = n\left\lfloor \frac{x}{n} \right\rfloor.$$

Their interval extensions are

$$\text{Ceil}^*([l,u],n) = [\text{ceil}_n(l), \text{ceil}_n(u)],$$

and

$$\text{Floor}^*([l,u],n) = [\text{floor}_n(l), \text{floor}_n(u)].$$

Comparison primitive. For a reachable target program point, relational intents are evaluated using

$$\text{Cmp}^* \;:\; \mathscr{I} \times \text{Op} \times \mathscr{I} \;\to\; \mathbb{J}_r,$$

where $\mathbb{J}_r = \{S, V, W\}$.

For left- and right-operand intervals L and R, respectively,

$$\text{Cmp}^*(L, op, R) = \begin{cases} S & \text{if } \forall l \in L, \forall r \in R : l \; op \; r, \\ V & \text{if } \forall l \in L, \forall r \in R : \neg(l \; op \; r), \\ W & \text{otherwise.} \end{cases}$$

Thus, S and V represent uniform satisfaction and uniform violation, respectively, over

all concrete value combinations represented by the two interval operands. If neither condition can be established, the comparison yields W. For $op \in \{\texttt{in}, \texttt{not in}\}$, set membership over the values represented by the intervals replaces the pointwise ordering relation.

Truthiness implication primitive. The implication form $=>$ is interpreted according to the zero/non-zero status of its two interval operands:

$$\mathrm{Impl}^* \ : \ \mathscr{I} \times \mathscr{I} \ \to \ \mathbb{J}_r,$$

with

$$\mathrm{Impl}^*(L,R) = \begin{cases} S & \text{if } L = \{0\}, \\ S & \text{if } 0 \notin L \text{ and } 0 \notin R, \\ V & \text{if } 0 \notin L \text{ and } R = \{0\}, \\ W & \text{otherwise.} \end{cases}$$

Here, zero represents false and a non-zero value represents true. Accordingly, the implication is satisfied when the left operand is definitely false or when both operands are definitely non-zero. It is violated when the left operand is definitely non-zero while the right operand is definitely zero. All remaining cases yield W because the interval abstraction does not establish a definite truthiness relation.

Feasibility primitive. The feasibility forms use

$$\mathrm{Feasible}^* \ : \ \mathscr{B} \ \to \ \mathbb{J}_r, \qquad \mathrm{Feasible}^*(B) = \begin{cases} V & \text{if } B = \{0\}, \\ S & \text{otherwise.} \end{cases}$$

Unlike relational validation, feasibility asks whether the condition can hold for at least one concrete state represented by the abstract Boolean value. Thus, both $\{1\}$ and $\{0,1\}$ are classified as feasible, whereas only $\{0\}$ yields V.

Meaning function. The meaning function $\mathscr{V}[\![\cdot]\!]$, introduced in Section 4.3, is defined by case analysis over the intent forms. We use e for an intent-value expression, f for a function identifier, and ℓ_{ret} for the return-statement line selected by the intent. We further use $\mathrm{retEnv}(\ell_{\mathrm{ret}}, \Gamma)$ to denote the abstract variable environment at the corresponding return statement. Before applying the form-specific semantics, IQDebug checks whether the

target program point has a reachable abstract state. If the target point is unreachable under the current analysis context, the meaning function yields U. Otherwise, the corresponding intent form is evaluated using the semantic primitives defined above, yielding a judgment in $\mathbb{J}_r = S, V, W$. Thus, for a well-formed intent a,

$$\mathscr{V}[\![a]\!](\Gamma) = \begin{cases} U & \text{if the target program point is unreachable,} \\ \mathscr{V}_r[\![a]\!](\Gamma) & \text{otherwise,} \end{cases}$$

where $\mathscr{V}_r$ denotes the form-specific semantics over reachable abstract states defined below.

`@During` **intents.** The forms specific to `@During` are call-argument comparison and feasibility. Let $\mathscr{K}$ denote the statements associated with the selected source location. The n-th call argument is evaluated using the abstract state immediately before the call, while the feasibility of a `require` or `assert` condition is evaluated at the target program point:

$$\begin{aligned} \mathscr{V}_r[\![f.\mathrm{arg}[n]\ op\ e]\!](\Gamma) &= \mathrm{Cmp}^*\big(\mathrm{eval}(\sigma_{\mathrm{before}}, \mathrm{arg}_n(f, \mathscr{K})), op, \mathrm{eval}(\sigma_{\mathrm{before}}, e)\big), \\ \mathscr{V}_r[\![\text{require feasible}]\!](\Gamma) &= \mathrm{Feasible}^*\big(\mathrm{eval}_{\mathscr{B}}(\sigma_{\mathrm{pt}}, \mathrm{cond}_{\mathrm{req}}(\mathscr{K}))\big), \\ \mathscr{V}_r[\![\text{assert feasible}]\!](\Gamma) &= \mathrm{Feasible}^*\big(\mathrm{eval}_{\mathscr{B}}(\sigma_{\mathrm{pt}}, \mathrm{cond}_{\mathrm{asrt}}(\mathscr{K}))\big). \end{aligned}$$

`@Post` **intents.** All `@Post` intents use forms shared with `@During` and therefore require no additional scope-specific semantic equations. Their unqualified expressions are evaluated with respect to the target program-point state, which corresponds to the normal function-exit state under `@Post`.

Common-form intents. For forms that may appear under either intent scope, unqualified expressions are evaluated using the abstract state at the target program point:

$$\mathrm{ref}(\Gamma) = \sigma_{\mathrm{pt}}.$$

Under `@During`, σ_{pt} corresponds to the state after the selected statement has been interpreted, whereas under `@Post`, it corresponds to the normal function-exit state. The return-summary forms directly compare a captured return value:

$$\mathscr{V}_r[\![\text{returnExpression } op\ e]\!](\Gamma) = \mathrm{Cmp}^*\big(R(\ell_{\mathrm{ret}}), op, \mathrm{eval}(\mathrm{ref}(\Gamma), e)\big),$$
$$\mathscr{V}_r[\![\text{return}[n]\ op\ e]\!](\Gamma) = \mathrm{Cmp}^*\big(R(\ell_{\mathrm{ret}})[n], op, \mathrm{eval}(\mathrm{ref}(\Gamma), e)\big).$$

The `return` variable-comparison form instead evaluates both operands in the abstract environment associated with the selected return statement:

$$\mathscr{V}_r[\![\text{return } e_l\ op\ e_r]\!](\Gamma) = \mathrm{Cmp}^*\big(\mathrm{eval}(\mathrm{retEnv}(\ell_{\mathrm{ret}}, \Gamma), e_l), op, \mathrm{eval}(\mathrm{retEnv}(\ell_{\mathrm{ret}}, \Gamma), e_r)\big).$$

The three numeric-helper forms first transform the right-hand interval and then apply the ordinary comparison primitive:

$$\mathscr{V}_r[\![e_l\ op\ \texttt{PercentOf}(e_r, p)]\!](\Gamma) = \mathrm{Cmp}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_l), op, \mathrm{PercentOf}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_r), p\big)\big),$$
$$\mathscr{V}_r[\![e_l\ op\ \texttt{ceil}(e_r, n)]\!](\Gamma) = \mathrm{Cmp}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_l), op, \mathrm{Ceil}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_r), n\big)\big),$$
$$\mathscr{V}_r[\![e_l\ op\ \texttt{floor}(e_r, n)]\!](\Gamma) = \mathrm{Cmp}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_l), op, \mathrm{Floor}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_r), n\big)\big).$$

Ordinary relational comparison and truthiness implication are defined by

$$\mathscr{V}_r[\![e_l\ op\ e_r]\!](\Gamma) = \mathrm{Cmp}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_l), op, \mathrm{eval}(\mathrm{ref}(\Gamma), e_r)\big),$$
$$\mathscr{V}_r[\![e_l \Rightarrow e_r]\!](\Gamma) = \mathrm{Impl}^*\big(\mathrm{eval}(\mathrm{ref}(\Gamma), e_l), \mathrm{eval}(\mathrm{ref}(\Gamma), e_r)\big).$$

Change detection. The remaining common form, $\mathrm{changed}(e, b)$, compares the value of e at function entry with its value at the target program point. Because the target program-point state already represents the evaluation boundary for both intent scopes, we define

$$\mathrm{bound}(\Gamma) = \sigma_{\mathrm{pt}}.$$

Algorithm 6 Interval Comparison Diagnostics for Cmp*-based Intents

Input: Intervals L, R, comparison operator op, judgment $j \in \mathbb{J}_r$
Output: $(regions, shape)$
$regions \leftarrow ExtractViolatedRegion(L, R, op)$
if $j \neq W$ **then**
    return $(regions, \bot)$
end if
$shape \leftarrow ClassifyWarningShape(regions, op)$
return $(regions, shape)$

The two forms are then defined by ordinary equality and inequality comparison:

$$\mathscr{V}_r[\![\text{changed}(e, \text{true})]\!](\Gamma) = \text{Cmp}^*\big(\text{eval}(\sigma_{\text{entry}}, e), \neq, \text{eval}(\text{bound}(\Gamma), e)\big),$$

$$\mathscr{V}_r[\![\text{changed}(e, \text{false})]\!](\Gamma) = \text{Cmp}^*\big(\text{eval}(\sigma_{\text{entry}}, e), =, \text{eval}(\text{bound}(\Gamma), e)\big).$$

Thus, changed captures whether the value of e differs between function entry and the intent's target program point. It does not record whether e was modified at an intermediate program point and later restored to its entry value.

Interval Comparison Diagnostics

For intents whose reachable-state judgments are determined by Cmp*, IQDebug derives additional diagnostic information from the same abstract interval comparison. These diagnostics are computed after the semantic judgment has been determined and do not alter that judgment.

Algorithm 6 presents the procedure used by IQDebug to derive violated-region information and, for a *Warning* judgment, classify the directional structure of the resulting uncertainty. IQDebug derives two forms of auxiliary diagnostic information from a Cmp*-based comparison:

- **Violated region** — the portion of the left-operand interval L containing values for which the specified comparison can be violated by at least one value represented by the right operand R;
- **Warning shape** — for a *Warning* judgment, a coarse Type 1/Type 2/Type 3 classi-

fication describing the directional structure of the violated region.

These diagnostics are downstream of the semantic judgment $j \in \mathbb{J}_r$ and do not refine or alter the *Satisfied*, *Violated*, or *Warning* result. The diagnostics instead provide supplementary information for interpreting the interval comparison once the target program point has been determined to be reachable. The other semantic primitives require no dedicated interval-diagnostic procedure. Impl is determined directly from the zero/non-zero status of its interval operands, while Feasible is determined from the abstract Boolean value of the corresponding condition.

Step 1: Violated-region extraction (Algorithm 6, line 3). Let $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ be finite integer intervals. To identify which portions of the left operand contribute to a violated or inconclusive comparison, IQDebug partitions the values represented by L into three disjoint regions:

$$T_s = \{\, l \in L \mid \forall r \in R,\ l \ op\ r \,\},$$

$$T_v = \{\, l \in L \mid \forall r \in R,\ \neg(l \ op\ r) \,\},$$

and

$$T_w = L \setminus (T_s \cup T_v).$$

Here, T_s, T_v, and T_w characterize three regions of the abstract left-operand interval L with respect to R. T_s denotes the region whose values are guaranteed to satisfy the comparison against every value represented by R, whereas T_v denotes the region whose values are guaranteed to violate it. T_w contains the remaining values, for which neither uniform satisfaction nor uniform violation can be established with respect to R. Accordingly, T_s, T_v, and T_w provide a value-level decomposition of the same abstract comparison used to derive the semantic judgment.

Table 4.4 presents the *Satisfied* and *Violated* conditions for all supported comparison operators. For the ordered relational operators $>$, $\geq$, $<$, and $\leq$, the relative positions of the endpoints of L and R determine whether the relation holds or fails uniformly over the two abstract intervals. For $=$ and $\neq$, the result depends on the overlap between L and R and, in the singleton case, on whether both operands represent the same integer value. The membership operators `in` and `not in` are interpreted with respect to the sets

Table 4.4: Per-operator `Satisfied`/`Violated` semantic conditions.

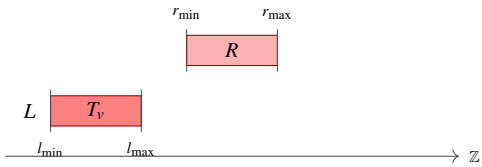
op	Satisfied condition	Violated condition
$>$	$l_{\min} > r_{\max}$	$l_{\max} \leq r_{\min}$
$\geq$	$l_{\min} \geq r_{\max}$	$l_{\max} < r_{\min}$
$<$	$l_{\max} < r_{\min}$	$l_{\min} \geq r_{\max}$
$\leq$	$l_{\max} \leq r_{\min}$	$l_{\min} > r_{\max}$
$=$	$L = R$, both singletons	$L \cap R = \emptyset$
$\neq$	$L \cap R = \emptyset$	$L = R$, both singletons
`in`	$L \subseteq R$	$L \cap R = \emptyset$
`not in`	$L \cap R = \emptyset$	$L \subseteq R$

of integer values represented by the intervals rather than through an ordering relation. Thus, `in` is *Satisfied* when $L \subseteq R$ and *Violated* when $L \cap R = \emptyset$, whereas `not in` uses the converse conditions. For every supported operator, a comparison that satisfies neither the *Satisfied* nor the *Violated* condition yields *Warning* under Cmp^*.

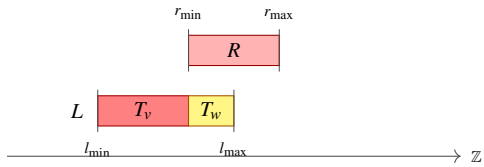
Figure 4.4 illustrates, for the representative operator $>$, how the left-operand interval L is partitioned into the satisfying region T_s, the violating region T_v, and the indeterminate region T_w according to the relative position of the right-operand interval R. Across subfigures (a)–(f), the figure covers the full spectrum from the case where L lies entirely below R to the case where L lies entirely above R. The color coding indicates which values of L are guaranteed to violate the comparison (T_v), guaranteed to satisfy it (T_s), or remain inconclusive under the abstraction (T_w).

In Figure 4.4(a), the entire interval L is located below R. Hence every value represented by L fails to satisfy $l > r$ for all $r \in R$, so $T_v = L$ and the comparison is *Violated*. At the opposite extreme, Figure 4.4(f) places the whole interval L above R. In this case every value in L is greater than every value in R, so $T_s = L$ and the comparison is *Satisfied*. The remaining four subfigures illustrate the different kinds of *Warning* configurations.

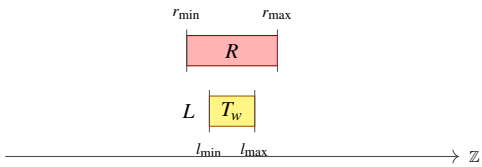
In Figure 4.4(b), L extends upward into the range of values that may exceed some elements of R but not all of them. Accordingly, L is divided into a definitely violating part T_v and an indeterminate part T_w, while T_s is empty. Figure 4.4(c) shows the case in



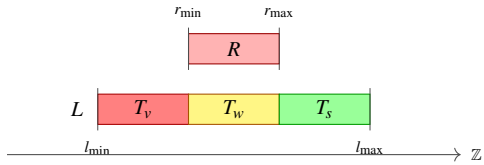
(a) L below R: $T_v = L$ (Violated).



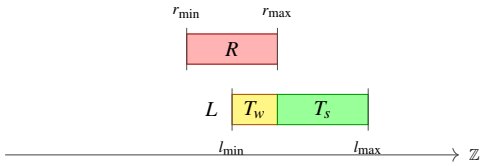
(b) L extends below R: $T_s = \emptyset$ (Warning).



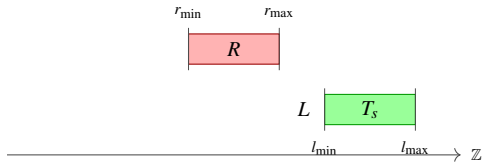
(c) L inside R: $T_w = L$ (Warning).



(d) L surrounds R (canonical): all three regions non-empty (Warning).



(e) L extends above R: $T_v = \emptyset$ (Warning).



(f) L above R: $T_s = L$ (Satisfied).

Figure 4.4: Spectrum of L vs R configurations for $op =$ "$>$".

which L lies entirely within R. No value of L is guaranteed to be greater than all values of R, and no value is guaranteed to be less than or equal to all values of R. Thus the whole interval is indeterminate, yielding $T_w = L$. Figure 4.4(d) is the canonical mixed case in which L surrounds R. Here all three regions are non-empty: lower values of L belong to T_v, upper values belong to T_s, and the overlapping middle portion belongs to T_w. Finally, Figure 4.4(e) is symmetric to Figure 4.4(b): the lower part of L is indeterminate, the upper part is guaranteed to satisfy the comparison, and no part belongs to T_v.

These subfigures together illustrate an important property of Cmp^*. A comparison is *Violated* only when the whole of L belongs to T_v, and it is *Satisfied* only when the whole of L belongs to T_s. Whenever L contains both definite and non-definite portions, or consists entirely of indeterminate values, the judgment becomes *Warning*. The figure therefore makes explicit how interval abstraction separates guaranteed satisfaction, guaranteed violation, and uncertainty within the same left-operand interval. Based on this

Table 4.5: Per-operator violated-region decomposition.

op	L_{lower}	L_{upper}	$L_{overlap}$
$>$	$[l_{\min}, \min(l_{\max}, r_{\min})]$	—	$[\max(l_{\min}, r_{\min}+1), \min(l_{\max}, r_{\max})]$
$\geq$	$[l_{\min}, \min(l_{\max}, r_{\min}-1)]$	—	$[\max(l_{\min}, r_{\min}), \min(l_{\max}, r_{\max}-1)]$
$<$	—	$[\max(l_{\min}, r_{\max}), l_{\max}]$	$[\max(l_{\min}, r_{\min}), \min(l_{\max}, r_{\max}-1)]$
$\leq$	—	$[\max(l_{\min}, r_{\max}+1), l_{\max}]$	$[\max(l_{\min}, r_{\min}+1), \min(l_{\max}, r_{\max})]$
$=$	$[l_{\min}, \min(l_{\max}, r_{\min}-1)]$	$[\max(l_{\min}, r_{\max}+1), l_{\max}]$	$\begin{cases} L \cap R & \text{if } \lvert R\rvert > 1, \\ \emptyset & \text{if } \lvert R\rvert = 1 \end{cases}$
$\neq$	—	—	$L \cap R$
`in`	$[l_{\min}, \min(l_{\max}, r_{\min}-1)]$	$[\max(l_{\min}, r_{\max}+1), l_{\max}]$	—
`not in`	—	—	$L \cap R$

partition, IQDebug reports the *violated region* as the portion of L for which a violating comparison remains possible:

$$V = \{\, l \in L \mid \exists r \in R,\ \neg(l\ op\ r) \,\}.$$

For the partition above, this region is exactly

$$V = T_v \cup T_w = L \setminus T_s.$$

Thus, the violated region includes both values that definitely violate the comparison and values for which violation remains possible under the abstract right operand. Only the satisfying region T_s, whose values are guaranteed to satisfy the comparison against all values represented by R, is excluded from the reported violated region.

Table 4.5 presents the per-operator decomposition of the violated region into lower, upper, and overlapping components. An interval whose lower endpoint exceeds its upper endpoint is treated as empty, and "—" indicates that no component exists in the corresponding direction.

For diagnostic reporting, IQDebug decomposes the violated region according to the position of its values relative to the interval represented by R:

- L_{lower} contains values in V that lie below the range represented by R;

- L_{upper} contains values in V that lie above the range represented by R;
- $L_{overlap}$ contains values in V that overlap the range represented by R.

Each component is either an integer sub-interval or empty, and together the three components reconstruct the violated region:

$$V = L_{lower} \cup L_{upper} \cup L_{overlap}.$$

The interpretation of these components depends on the comparison operator.

- **Ordered comparisons ($>$, $\geq$, $<$, and $\leq$).** The violated region is determined by the relative ordering of L and the endpoints of R. Depending on the direction of the comparison, L_{lower} or L_{upper} captures values that definitely violate the relation, while $L_{overlap}$ captures values for which violation remains possible because they overlap the range represented by R.
- **Equality and inequality ($=$ and $\neq$).** For equality, values outside R are definite violations and therefore appear in L_{lower} and L_{upper}. When R is a non-singleton interval, values in $L \cap R$ are also included in $L_{overlap}$, because equality is not guaranteed against every value represented by R. When R is a singleton, the matching value is guaranteed to satisfy the relation and is therefore excluded from the violated region. For inequality, only $L \cap R$ contributes to the violated region. If R is a singleton, the matching value definitely violates $\neq$; otherwise, overlapping values remain values for which violation is possible.
- **Membership comparisons (`in` and `not in`).** For `in`, values outside the set represented by R are definite violations and therefore appear in L_{lower} and L_{upper}. For `not in`, the converse applies: values in $L \cap R$ belong to the violated region and are represented by $L_{overlap}$.

Step 2: Warning-shape classification (Algorithm 6, lines 4–7). For a *Warning* judgment, IQDebug further summarizes the directional structure of the violated region derived in Step 1. The classification is based on whether violating values occur below, above, or within the range represented by R, using L_{lower}, L_{upper}, and $L_{overlap}$.

- **Type 1 (overlap-only).** Both L_{lower} and L_{upper} are empty. Accordingly, the violated region is confined to $L_{overlap}$, indicating that possible violation arises only within the range represented by R.
- **Type 2 (one-sided).** Exactly one of L_{lower} and L_{upper} is non-empty. The violated region therefore extends to only one side of the range represented by R, possibly together with an overlapping component.
- **Type 3 (two-sided).** Both L_{lower} and L_{upper} are non-empty. In this case, the violated region extends to both sides of the range represented by R, possibly together with an overlapping component.

These warning types characterize only the geometric structure of the abstract violated region. They do not constitute a severity, exploitability, likelihood, or priority ordering. For example, a Type 3 warning is not inherently more severe than a Type 2 or Type 1 warning; its practical significance depends on the semantics of the program and the surrounding protocol logic.

Accordingly, IQDebug separates semantic validation from diagnostic classification. The *Satisfied*, *Violated*, and *Warning* judgments are determined solely by the abstract comparison semantics. The violated region and the Type 1–3 warning shape are derived afterward as auxiliary diagnostics to help developers interpret why a *Warning* arises and how the possible violating values are distributed relative to R.

4.4 Source-Level Debugging over Abstract Program States

IQDebug provides source-level debugging by navigating abstract program states rather than replaying a concrete transaction trace. As described in Section 4.2, abstract interpretation computes an abstract environment for each source-level program point and stores the resulting state in the corresponding CFG node. The line-to-node mapping connects source locations to these CFG nodes, allowing the debugger to associate a selected source line with the abstract program state computed at that point. The state presented by IQDebug therefore represents an abstraction of the program values that may occur at the selected program point rather than the values from a single concrete execution. Numerical variables are represented by intervals, while supported non-numerical

values are represented using their corresponding abstract domains. Also, when an intent is associated with the current program point, the debugger can additionally present the validation judgment and, where applicable, the diagnostic information derived in Section 4.3. Thus, the same source-level program point serves as the common reference for inspecting abstract values and interpreting developer-authored intent.

Navigation itself does not invoke the abstract interpreter as long as the analyzed source code and debugging inputs remain unchanged. Instead, IQDebug reuses the abstract states already stored in the CFG and moves among source-level program points according to the CFG structure. Consequently, a step operation in IQDebug does not execute the next concrete instruction. It navigates to the next relevant source-level program point and presents the abstract state associated with that point. In this respect, IQDebug differs from a conventional debugger that follows a single concrete execution trace: multiple feasible control-flow successors or predecessors may be represented simultaneously by the abstract CFG.

Loop states require a similar interpretation. IQDebug does not maintain a sequence of concrete states for individual loop iterations. The abstract state associated with a loop body represents the state stabilized through fixpoint computation and therefore summarizes the loop behavior rather than a particular iteration. The navigation behavior for loops is described in Section **??**. **Update handling and state reuse.** Because source code, debugging inputs, and developer intent can be modified independently during debugging, IQDebug distinguishes three update cases and reuses existing analysis artifacts whenever possible.

- **Intent-only update.** When only the developer intent changes, the program structure and its abstract states remain unchanged. IQDebug therefore reuses both the existing CFG and the abstract states stored in its nodes and evaluates the new intent against the already available validation context. Neither source parsing nor abstract interpretation is required.

- **Intent and debugging-input update.** When the debugging inputs change, the source-level program structure remains unchanged and the existing CFG can therefore be reused. However, the updated inputs may change the initial abstract environment of the analyzed function and consequently affect its downstream abstract

states. IQDebug reinterprets the target function from its entry using the updated debugging inputs, updates the abstract states stored in the CFG, and subsequently reevaluates the intent over the resulting validation context.

- **Source-code update.** When the source code changes, IQDebug does not reconstruct and reinterpret the entire program from scratch. Instead, it applies the incremental source-processing and CFG-update procedure described in Section 4.2. The affected CFG portion is updated, abstract interpretation resumes from the affected program point, and downstream states are recomputed as necessary. Intent validation and subsequent debugging navigation then operate on the updated abstract states.

IQDebug therefore separates validation-only updates, interpretation-only updates, and structural source updates so that each form of interaction triggers only the backend processing required by the changed information.

4.4.1 Forward and Backward Navigation

IQDebug provides bidirectional source-level navigation over the feasible control-flow relationships represented in the CFG. Because navigation operates over abstract program states rather than a concrete execution trace, forward and backward steps may expose multiple feasible control-flow directions at the same program point.

Step Forward. A forward step examines the feasible successor nodes of the current CFG node. If exactly one feasible successor exists, the cursor moves to that successor and IQDebug presents its associated abstract state. If multiple feasible successors exist, IQDebug displays all of them simultaneously rather than selecting a single concrete path. Successors determined to be unreachable by abstract interpretation are excluded from the feasible successor set. Accordingly, at a branch point, the debugger exposes all control-flow directions that remain feasible under the current abstract analysis context.

Step Backward. A backward step is defined over the feasible predecessors of the current CFG node and does not rely on navigation history. If exactly one feasible predecessor exists, the cursor moves to that predecessor. If multiple feasible predecessors exist, the cursor moves to the first feasible predecessor while IQDebug highlights all feasible pre-

decessors simultaneously. The cursor position therefore serves only as the current navigation focus and does not imply that the highlighted predecessor was the unique path by which the current node was reached. Backward navigation thus exposes the possible predecessor states of the current program point rather than reconstructing a unique execution history.

4.4.2 Function-Call Navigation

IQDebug supports source-level navigation across function-call boundaries when the callee CFG can be determined from the available program representation and abstract analysis context.

Step Into. A *Step Into* operation moves the debugging focus from a call site to the entry of the corresponding callee CFG. IQDebug supports this operation for same-contract internal calls, inherited function calls, library calls, and cross-contract calls whose target can be statically associated with an available source-level contract representation. For cross-contract navigation, the callee must be identifiable from the source-level contract object and its corresponding dependency CFG must be available. For example, if a variable is declared with a concrete contract type, such as `Token token;` `token.transfer(...);`, and the CFG of `Token` is available, IQDebug can navigate into the corresponding callee function. In contrast, address-based or dynamically dispatched calls for which no unique source-level callee can be determined remain opaque to *Step Into*. Low-level calls such as `call`, `delegatecall`, `staticcall`, and native address-based `transfer` are therefore not navigable unless a unique source-level callee representation is otherwise available.

Interface-only external calls. When only an interface declaration is available and the corresponding implementation CFG is unavailable, IQDebug cannot navigate into the callee because no source-level program representation exists for the external implementation. Such a call is therefore treated as an opaque external call. If an `@IReturn` debugging input is provided, IQDebug can use the supplied return value to continue abstract interpretation on the caller side. However, `@IReturn` does not provide a callee CFG and therefore does not enable *Step Into*. It supports continuation of caller-side analysis rather than navigation inside the external call.

Step Out. A *Step Out* operation returns the debugging focus from the current callee CFG to the caller-side program point following the call site. The abstract state presented after returning reflects the return value and state effects propagated from the analyzed callee.

4.4.3 Breakpoint-Based Navigation

IQDebug supports breakpoint-based navigation over source-level program points represented in the CFG. A breakpoint is attached to a source line and mapped to the corresponding CFG node through the line-to-node mapping described in Section **??**. Unlike a breakpoint in a concrete runtime debugger, an IQDebug breakpoint does not suspend program execution. Instead, it serves as a navigation target over precomputed abstract program states.

Breakpoint. When a breakpoint is set on a source line, IQDebug associates it with the corresponding CFG node. If the target node is reachable under the current abstract analysis context, the breakpoint can be used as a navigation destination. If the target node is unreachable, the breakpoint remains visible but is marked as unreachable so that the developer can distinguish an inactive navigation target from a reachable one.

Continue. The *Continue* operation traverses the CFG forward from the current node and stops at the first reachable breakpoint encountered according to the debugger's CFG traversal order. Unreachable breakpoint nodes are skipped. When traversal reaches a branch, only feasible successor relationships identified by the abstract analysis are considered. Thus, *Continue* follows the same feasibility information used by ordinary forward navigation rather than executing a single concrete path.

Next and Previous Breakpoint. The *Next Breakpoint* and *Previous Breakpoint* operations reuse the same CFG traversal ordering used by *Continue*. *Next Breakpoint* moves to the next reachable breakpoint in that ordering, whereas *Previous Breakpoint* moves to the preceding reachable breakpoint. These operations are defined over CFG structure and are independent of navigation history.

Consequently, breakpoint-based navigation provides coarse-grained movement among developer-selected source locations while preserving the same abstract reachability semantics used by step-wise navigation.

4.4.4 Navigation over Control-Flow Structures

IQDebug adapts source-level navigation to control-flow structures whose abstract execution may represent multiple concrete paths or repeated executions. Rather than selecting a single concrete trace, the debugger preserves the feasible control-flow relationships identified by abstract interpretation and presents the corresponding abstract states at each program point.

Branches and joins. At a conditional branch, IQDebug considers only successor nodes whose abstract states remain feasible after condition pruning. A successor whose state is reduced to $\bot$ is treated as unreachable and is excluded from navigation. If more than one successor remains feasible, IQDebug displays all feasible successors simultaneously instead of selecting one as the active execution path. At a join point, the abstract state stored in the join node is obtained by joining the states propagated from all feasible predecessors. The resulting state therefore represents a merged abstraction of multiple possible incoming paths rather than the state of a single execution. During backward navigation from such a node, IQDebug highlights all feasible predecessor nodes. The cursor is placed on the first feasible predecessor only as a navigation focus; this position does not indicate that the program reached the join through that predecessor in a concrete execution.

Loops. IQDebug does not provide iteration-by-iteration navigation for loops. The abstract states associated with loop nodes are obtained through fixpoint computation and summarize the values that may arise across the represented iterations. Accordingly, a state displayed within a loop should be interpreted as a fixpoint-stabilized loop abstraction rather than as a snapshot of a particular iteration. A *Step Forward* operation from a loop header or a statement within the loop treats the loop as a summarized abstract transition. Instead of stepping through individual iterations, IQDebug moves the debugging focus to the first source-level statement following the loop and presents the corresponding post-loop abstract state. This avoids assigning a concrete iteration order to states that are represented only through the loop fixpoint. To expose how the loop changes the analyzed state, IQDebug additionally reports a loop-level delta at the loop header. The delta compares the abstract state before entering the loop with the fixpoint-stabilized state and reports only variables whose abstract values differ between the two. For example, a loop

may be summarized as

$$\texttt{acc} : [1,1] \to [1,\top], \qquad \texttt{i} : [0,0] \to [0,\top].$$

This information is a summary of the loop's abstract effect and should not be interpreted as an iteration trace.

Through these navigation rules, IQDebug preserves the abstract semantics of branches, joins, and loops while providing source-level movement without imposing a single concrete execution path.

Chapter 5. Evaluation

5.1 Experimental Setup

5.2 Intent Validation Capability

5.3 Specification and Analysis Cost

5.3.1 Specification Cost

5.3.2 Validation and Interactive Analysis Cost

5.4 Comparison with Existing Detection Tools

5.5 Threats to Validity

Chapter 6. Discussion

6.1 Design Trade-offs of Source-Level Debugging

6.2 Practical Guidelines for Debug Inputs and Intent Specification

6.3 Current Limitations

Chapter 7. Conclusion

Bibliography

[1] Sanidhay Arora et al. SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS)*, pages 1394–1405, 2024. doi: 10.1145/3634737.3637681.

[2] Massimo Bartoletti et al. Solvent: Liquidity verification of smart contracts. In *International Conference on Integrated Formal Methods (iFM)*, pages 256–266, 2024. doi: 10.1007/978-3-031-76554-4_14.

[3] Cristian Cadar et al. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008. doi: 10.5555/1855741.1855756.

[4] Jiachi Chen et al. NumScout: Unveiling numerical defects in smart contracts using LLM-pruning symbolic execution. *IEEE Transactions on Software Engineering*, 2025. doi: 10.1109/TSE.2025.3555622.

[5] Bharti Chimdyalwar. Fast and precise interval analysis on industry code. In *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 2024. doi: 10.1109/ISSREW63542.2024.00049.

[6] Consensys Diligence. Mythril. https://github.com/ConsenSysDiligence/mythril, 2026. Accessed April 2026.

[7] Consensys Diligence. MythX. https://mythx.io/, 2026. Accessed April 2026.

[8] Consensys Diligence. Scribble. https://diligence.security/scribble/, 2026. Accessed April 2026.

[9] Julian Erhard et al. Interactive abstract interpretation: reanalyzing multithreaded C programs for cheap. *International Journal on Software Tools for Technology Transfer*, 2024. doi: 10.1007/s10009-024-00768-9.

[10] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15, 2019. doi: 10.1109/WETSEB.2019.00008.

[11] Foundry Forge. Foundry Forge. https://book.getfoundry.sh/reference/forge/forge/, 2025. Accessed November 2025.

[12] Arie Gurfinkel et al. The SeaHorn verification framework. In *International Conference on Computer Aided Verification (CAV)*, pages 343–361, 2015. doi: 10.1007/978-3-319-21690-4_20.

[13] Hardhat. Hardhat. https://hardhat.org/, 2025. Accessed November 2025.

[14] IntentChecker. DAppSCAN contract analysis dataset. https://github.com/iwwyou/IntentChecker/blob/main/evaluation/DappSCAN_contract_analysis/contract_analysis_table.xlsx, 2026. Accessed April 2026.

[15] Inseong Jeon et al. SolQDebug: Debug Solidity quickly for interactive immediacy in smart contract development. *Automated Software Engineering*, 33(3):86, 2026. doi: 10.1007/s10515-026-00618-9.

[16] Queping Kong et al. DeFiTainter: Detecting price manipulation vulnerabilities in DeFi protocols. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1144–1156, 2023. doi: 10.1145/3597926.3598124.

[17] Xiaoli Li et al. Penetrating the hostile: Detecting DeFi protocol exploits through cross-contract analysis. *IEEE Transactions on Information Forensics and Security*, 20:11759–11774, 2025. doi: 10.1109/TIFS.2025.3626979.

[18] LLVM. libFuzzer. https://llvm.org/docs/LibFuzzer.html, 2026. Accessed April 2026.

[19] Tai D. Nguyen et al. An idealist's approach for smart contract correctness. In *International Conference on Formal Engineering Methods (ICFEM)*, pages 11–28,

2023. doi: 10.1007/978-981-99-7584-6_2.

[20] Sina Ramezany, Rachsuda Setthawong, and Pisal Setthawong. Midnight: An efficient event-driven EVM transaction security monitoring approach for flash loan detection. In *Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, pages 237–241, 2023. doi: 10.1109/JCSSE58229.2023.10201970.

[21] Remix IDE. Remix IDE. https://remix.ethereum.org/, 2025. Accessed November 2025.

[22] Idriss Riouak et al. IntraJ: an on-demand framework for intraprocedural Java code analysis. *International Journal on Software Tools for Technology Transfer*, 2024. doi: 10.1007/s10009-024-00771-0.

[23] Solidity. Solidity. https://www.soliditylang.org/, 2026. Accessed September 2026.

[24] Solidity Debugger Pro. Solidity Debugger Pro. https://www.soliditydbg.org/, 2025. Accessed November 2025.

[25] Majd Soud, Grischa Liebel, and Mohammad Hamdaqa. A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. *Empirical Software Engineering*, 29(1):13, 2024. doi: 10.1007/s10664-023-10398-5.

[26] Benno Stein, Bor-Yuh Evan Chang, and Manu Sridharan. Demanded abstract interpretation. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021. doi: 10.1145/3453483.3454044.

[27] Benno Stein, Bor-Yuh Evan Chang, and Manu Sridharan. Interactive abstract interpretation with demanded summarization. *ACM Transactions on Programming Languages and Systems*, 2024. doi: 10.1145/3648484.

[28] Yuqiang Sun et al. GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13, 2024. doi: 10.1145/3597503.3639117.

[29] Dabao Wang et al. DeFiGuard: A price manipulation detection service in DeFi using graph neural networks. *IEEE Transactions on Services Computing*, 2024. doi: 10.1109/TSC.2024.3489439.

[30] Sally Junsong Wang, Kexin Pei, and Junfeng Yang. SmartInv: Multimodal learning for smart contract invariant inference. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, pages 2217–2235, 2024. doi: 10.1109/SP54263.2024.00126.

[31] Scott Wesley et al. Verifying Solidity smart contracts via communication abstraction in SmartACE. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 425–449, 2022. doi: 10.1007/978-3-030-94583-1_21.

[32] Siwei Wu et al. DeFiRanger: Detecting DeFi price manipulation attacks. *IEEE Transactions on Dependable and Secure Computing*, 21(4):4147–4161, 2023. doi: 10.1109/TDSC.2023.3346888.

[33] Rongwei Xi, Zhuo Wang, and Karthik Pattabiraman. POMABuster: Detecting price oracle manipulation attacks in decentralized finance. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, pages 3923–3942, 2024. doi: 10.1109/SP54263.2024.00257.

[34] Brian Zhang. Towards finding accounting errors in smart contracts. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13, 2024. doi: 10.1145/3597503.3639128.

[35] Zhuo Zhang et al. Demystifying exploitable bugs in smart contracts. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 615–627, 2023. doi: 10.1109/ICSE48619.2023.00061.

[36] Zibin Zheng et al. DAppSCAN: Building large-scale datasets for smart contract weaknesses in dApp projects. *IEEE Transactions on Software Engineering*, 50(6):1360–1373, 2024. doi: 10.1109/TSE.2024.3383422.

[37] Liyi Zhou et al. SoK: Decentralized finance (DeFi) attacks. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*, pages 2444–2461, 2023. doi: 10.1109/SP46215.2023.10179435.

Appendix A. Incremental CFG Construction Patterns

This appendix provides the complete CFG construction patterns for IQDebug's incremental CFG builder, which follows the construction scheme introduced in SolQDebug [15]. The patterns show how each statement type is incrementally spliced into the existing control-flow graph.

A.1 CFG Hierarchy

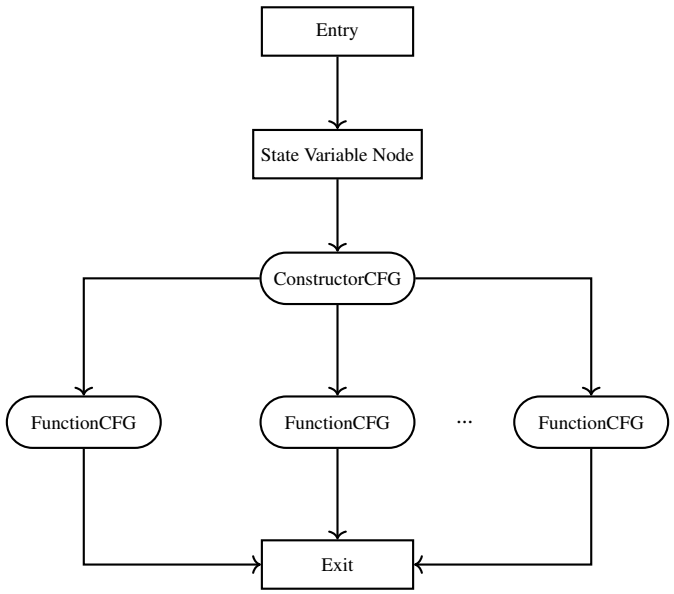


Figure A.1: Structure of ContractCFG. The contract-level CFG sequences state variable initialization, constructor execution, and all function definitions.

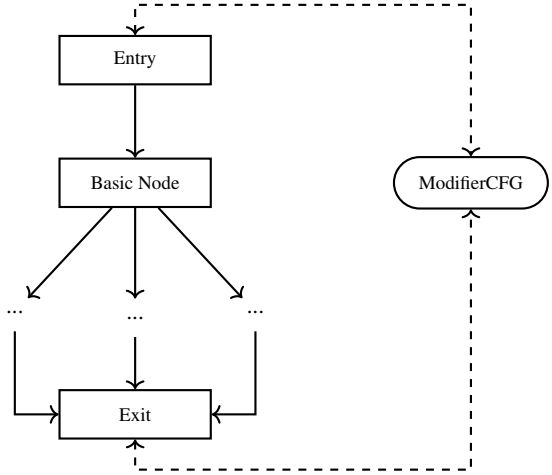


Figure A.2: Structure of FunctionCFG. Each function maintains its own control-flow graph. When modifiers are present, control flows from the function entry to the modifier, and returns from the modifier's placeholder (_) back to the function body, eventually reaching the function exit.

A.2 Statement-Level Construction Patterns

The following patterns show how each statement type is incrementally spliced into the existing control-flow graph. All patterns assume an initial state with a current node connected to successors (. . .), as illustrated in the simple statement example.

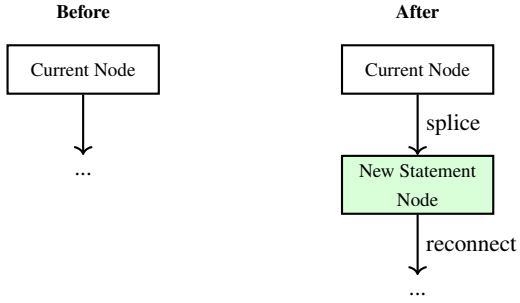


Figure A.3: Simple statement insertion: single node spliced between current node and successors

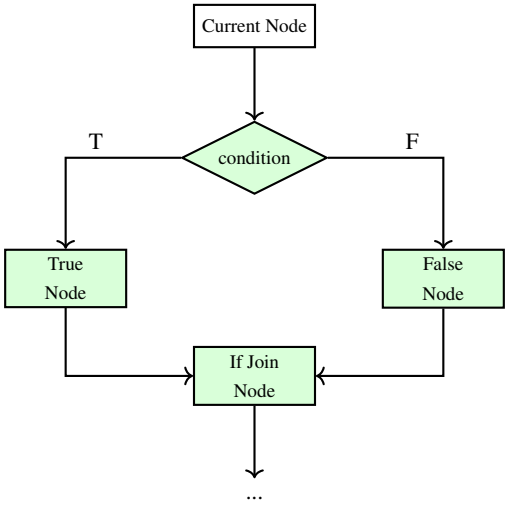


Figure A.4: If statement insertion. The builder creates a `condition node`, two nodes for true/false arms, and an `if join`

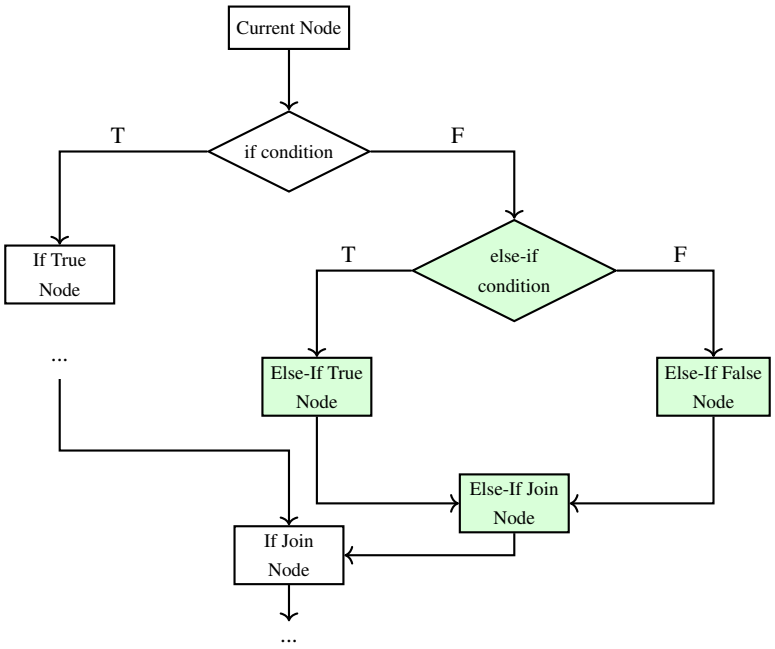


Figure A.5: Else-if statement insertion. The builder replaces the false arm with a new `condition node`, two nodes, and an `else-if join`

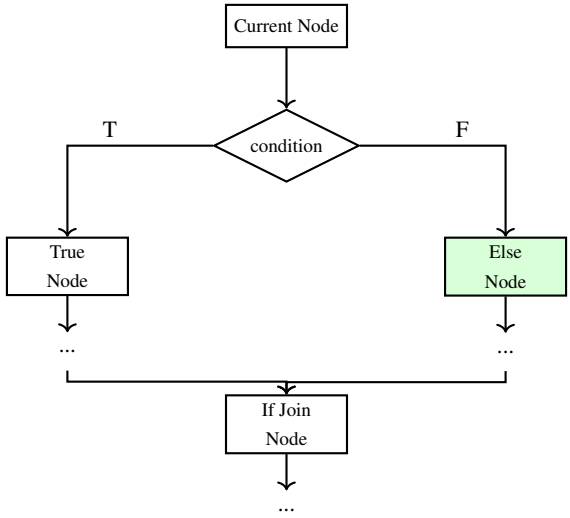


Figure A.6: Else statement insertion. The builder replaces the false branch node with a new else node, connecting to the `if join`

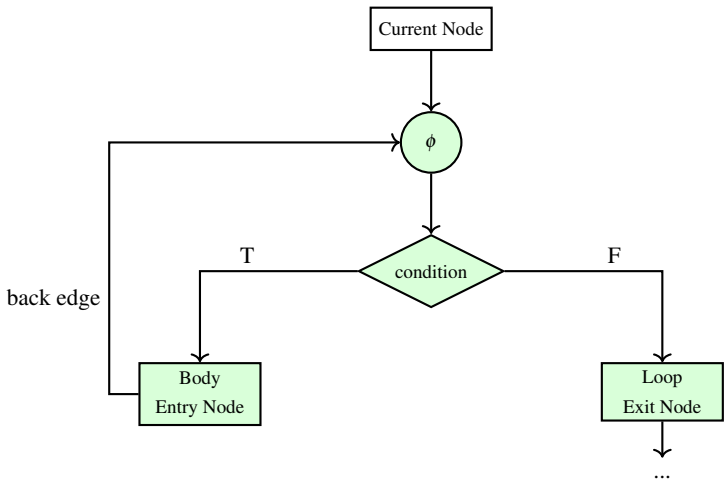


Figure A.7: While loop insertion. The builder creates a `fixpoint evaluation node` ϕ, a `condition node`, a loop body node, and a `loop exit node`

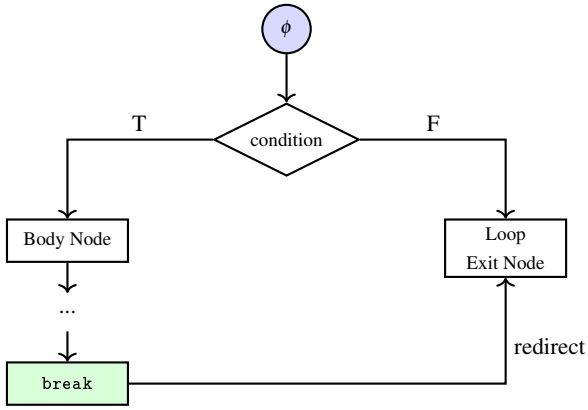


Figure A.8: Break statement insertion. The `break` node's outgoing edge is redirected to the `loop exit node`

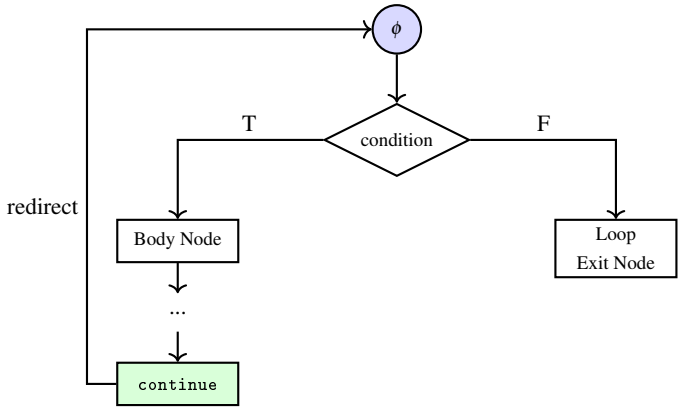


Figure A.9: Continue statement insertion. The `continue` node's outgoing edge is redirected to the loop's `fixpoint evaluation node` ϕ

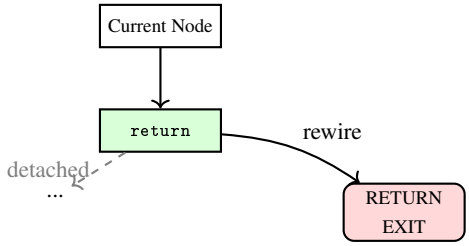


Figure A.10: Return statement insertion. The containing node's edge is rewired to the function's unique `return exit node`

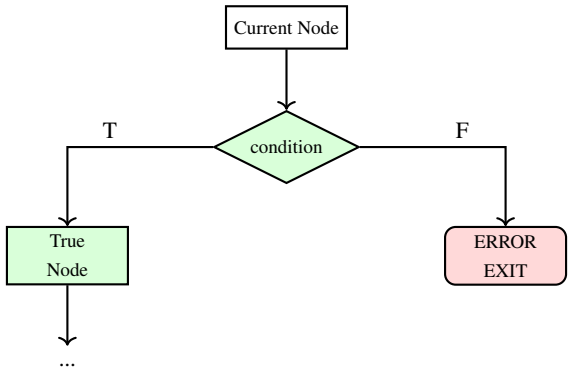


Figure A.11: Require/assert statement insertion. The builder creates a `condition node` with true edge to a node and false edge to the `error exit node`

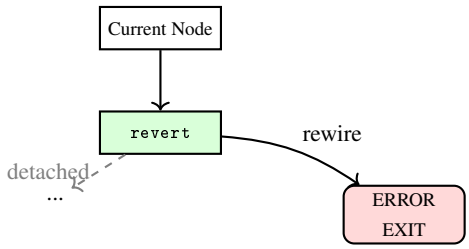


Figure A.12: Revert statement insertion. The containing node's edge is rewired to the function's unique `error exit node`

Appendix B. Abstract Domain and Formal Semantics

This appendix presents the abstract domains and formal semantics underlying IQDebug's abstract interpreter. The framework uses interval abstractions for numerical values, finite-set abstractions for addresses, and structured abstract representations for composite Solidity values. These definitions provide the semantic foundation for the program-state computation described in Section 4.2.3.

B.1 Language Syntax

We consider the subset of Solidity expressions and statements required by IQDebug's source-level analysis. The syntax abstracts away concrete Solidity parsing details while retaining the constructs relevant to value propagation and control flow.

Expressions.

$$\begin{aligned} e \in \mathsf{Expr} \ ::= \ & n \mid x \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{address_literal} \\ & \mid e_1 \oplus e_2 \mid e_1 \ \odot \ e_2 \mid e_1 ? e_2 : e_3 \\ & \mid e.f \mid e_1[e_2] \mid f(\overline{e}) \mid \neg e \mid \mathsf{delete}\ e \end{aligned}$$

where $\oplus \in \{+, -, *, /, \%, **, \&\&, ||, \&, |, \wedge, <<, >>\}$ and $\odot \in \{<, \leq, >, \geq, ==, \neq\}$.

Statements.

$$
\begin{aligned}
s \in \mathsf{Stmt} \ ::= \ & \mathsf{skip} \mid s_1; s_2 \mid \tau\, x; \mid \tau\, x = e; \\
& \mid lv := e \mid \mathsf{delete}\ lv \\
& \mid \mathsf{if}\ p\ \mathsf{then}\ s_t\ \mathsf{else}\ s_f \\
& \mid \mathsf{while}\ p\ \mathsf{do}\ s \\
& \mid \mathsf{for}\ init; p; incr\ \mathsf{do}\ s \\
& \mid \mathsf{do}\ s\ \mathsf{while}\ p \\
& \mid \mathsf{return}\ e \mid \mathsf{assert}(p) \mid \mathsf{require}(p) \\
& \mid \mathsf{revert}(\cdots) \\
& \mid \mathsf{try}\ e\ (\mathsf{returns}(x))\ s_t\ \mathsf{catch}\ s_c \\
& \mid f(\overline{e})
\end{aligned}
$$

Here, τ ranges over supported Solidity types, including integer, Boolean, address, array, mapping, and struct types. The metavariable lv denotes an assignable location such as a variable, struct field, array element, or mapping entry, while p denotes a Boolean predicate.

B.2 Abstract Domain

IQDebug represents program variables using type-specific abstract domains. For numerical values, intervals summarize sets of concrete values, while other Solidity types use abstractions appropriate to their semantics.

Atomic abstract values.

- **Unsigned integers:**

$$\widehat{\mathbb{U}}_N = \{[\ell, u] \mid 0 \le \ell \le u \le 2^N - 1\} \cup \{\bot, \top_N^{\mathbb{U}}\}.$$

- **Signed integers:**

$$\widehat{\mathbb{Z}}_N = \{[\ell, u] \mid -2^{N-1} \le \ell \le u \le 2^{N-1} - 1\} \cup \{\bot, \top_N^{\mathbb{Z}}\}.$$

- **Booleans:**
$$\widehat{\mathbb{B}} = \{\bot, \widehat{\mathsf{false}}, \widehat{\mathsf{true}}, \top\}.$$

- **Addresses:**
$$\widehat{\mathbb{A}} = \{\bot\} \cup \wp_{\text{fin}}(\mathsf{AddrID}) \cup \{\top\},$$
where finite sets contain symbolic address identifiers.

- **Bytes and strings:** Values whose concrete contents are not materialized are represented symbolically or by an opaque abstract value.

- **Enums:**
$$\widehat{\mathsf{Enum}}(E) = \{[\ell, u] \mid 0 \le \ell \le u \le |E| - 1\} \cup \{\bot, \top_E\}.$$

Composite abstract values.

- **Structs:**
$$\widehat{\mathsf{Struct}}(C) = \prod_{f \in \mathsf{fields}(C)} \widehat{\mathsf{Val}}_f,$$
with component-wise ordering and updates.

- **Arrays:**
$$\widehat{\mathsf{Arr}}(\tau) = (\hat{\ell}, \hat{d}, M),$$
where $\hat{\ell} \in \widehat{\mathbb{U}}_{256}$ represents the array length, $\hat{d}$ is the default abstract element, and $M : \mathbb{N} \rightharpoonup \widehat{\tau}$ stores materialized indices.

- **Mappings:**
$$\widehat{\mathsf{Map}}(\kappa \Rightarrow \tau) = (\hat{d}, M),$$
where $\hat{d}$ is the default abstract value and M stores materialized keys.

Standard abstract-domain operations, including partial order, join ($\sqcup$), meet ($\sqcap$), widening, and narrowing, are defined over the corresponding domains. For integer and enum values, these operations follow the standard interval-domain definitions.

The abstract environment used by IQDebug is defined as

$$\hat{\sigma} \in \widehat{\Sigma} = \mathsf{Var} \rightharpoonup \widehat{\mathsf{Val}},$$

where each program variable is mapped to its corresponding abstract value.

B.3 Concrete Semantics

Let

$$\sigma \in \Sigma = \mathsf{Var} \rightharpoonup \mathsf{Val}$$

denote a concrete program store. Concrete execution maps statements and expressions over these stores according to standard Solidity semantics.

The semantic result of a statement is one of the following forms:

- $\mathsf{Norm}(\sigma)$: normal execution with resulting store σ;
- $\mathsf{Ret}(v, \sigma)$: function return with value v and store σ;
- Abort: abnormal termination caused by constructs such as `revert` or a failed `require`/`assert`.

The concrete semantics define the reference behavior from which the corresponding abstract semantics are derived. Assignments update the concrete store, conditionals select a branch according to their predicates, and loop semantics are expressed through least fixpoints. Function calls bind parameters to arguments for internal calls, while external-call behavior depends on the available callee representation.

B.4 Collecting Semantics

Abstract interpretation approximates sets of concrete executions rather than a single concrete execution. We therefore lift the concrete semantics to sets of stores.

For a function f with concrete semantics

$$[\![f]\!] : \Sigma \times \mathsf{Val} \to \mathit{Result},$$

the collecting semantics over a set of input states S is defined as

$$\mathscr{S}[[f]](S) = \{\sigma' \mid \exists \sigma \in S,\, v_{\text{in}} \in \mathsf{Val}.\, [\![f]\!](\sigma, v_{\text{in}}) = \mathsf{Norm}(\sigma') \\ \vee [\![f]\!](\sigma, v_{\text{in}}) = \mathsf{Ret}(v_{\text{out}}, \sigma')\}.$$

The collecting semantics characterizes the concrete states reachable under all represented inputs and execution paths. IQDebug approximates this potentially large or unbounded state set using the abstract domains defined above.

B.4.1 Abstract Semantics

IQDebug's abstract semantics safely over-approximate the collecting semantics through the abstraction and concretization functions α and γ. For each supported concrete domain, the corresponding abstract domain represents a set of concrete values, allowing the analyzer to propagate multiple concrete executions within a finite abstract representation.

The abstract semantic store is

$$\hat{\sigma} \in \widehat{\Sigma} = \mathsf{Var} \rightharpoonup \widehat{\mathsf{Val}}.$$

The analysis additionally uses the following auxiliary operations:

- $\text{refine}(\hat{\sigma}, p, b)$ restricts an abstract environment according to predicate p and branch outcome b;
- $\widehat{\text{write}}(\hat{\sigma}, lv, \hat{v})$ updates an abstract location, using a strong update when the target is uniquely identified and a weak update otherwise;
- $\text{joinRes}(r_1, r_2)$ combines abstract results from alternative execution paths.

Sequential statements apply their corresponding abstract transfer functions to the incoming environment. For conditionals, the incoming state is refined separately for the true and false branches and the resulting states are subsequently joined. Loop constructs are interpreted through fixpoint computation with widening and narrowing as described

in Section 4.2.3. Assignments and state updates operate over abstract values, while return and exceptional control-flow constructs produce their corresponding abstract result forms.

External calls are analyzed using the corresponding callee CFG when it is available. When an external implementation is unavailable, the abstract semantics conservatively represent information that cannot be derived from the available source-level representation. Developer-supplied debugging inputs, including `@IReturn`, may provide additional caller-side values as described in Section 4.2.2.

Table B.4: Concrete semantics (denotational)

Statement	**Meaning**
skip	$[\![\mathsf{skip}]\!](\sigma) = \mathsf{Norm}(\sigma)$
$s_1; s_2$	$[\![s_1; s_2]\!](\sigma) = \big([\![s_1]\!](\sigma)\big) \triangleright (\lambda\sigma'.\, [\![s_2]\!](\sigma'))$
$\tau\ x;$	$[\![\tau\ x;]\!](\sigma) = \mathsf{Norm}\big(\sigma[x \mapsto \mathrm{zero}_\tau]\big)$
$\tau\ x = e;$	$[\![\tau\ x = e;]\!](\sigma) = \mathsf{Norm}\big(\sigma[x \mapsto [\![e]\!]_\sigma]\big)$
$lv := e$	$[\![lv := e]\!](\sigma) = \mathsf{Norm}\big(\mathrm{write}(\sigma, \mathrm{loc}_\sigma(lv), [\![e]\!]_\sigma)\big)$
delete lv	$[\![\mathsf{delete}\ lv]\!](\sigma) = \mathsf{Norm}\big(\mathrm{write}(\sigma, \mathrm{loc}_\sigma(lv), \mathrm{zero}_{\tau(lv)})\big)$
if p then s_t else s_f	$[\![\cdot]\!](\sigma) = \begin{cases} [\![s_t]\!](\sigma) & \text{if } [\![p]\!]_\sigma = \mathsf{true}, \\ [\![s_f]\!](\sigma) & \text{if } [\![p]\!]_\sigma = \mathsf{false} \end{cases}$
while p do s	$F(H)(\sigma) = \begin{cases} \big([\![s]\!](\sigma)\big) \triangleright H & \text{if } [\![p]\!]_\sigma = \mathsf{true}, \\ \mathsf{Norm}(\sigma) & \text{if } [\![p]\!]_\sigma = \mathsf{false} \end{cases}$; $[\![\mathsf{while}\ p\ \mathsf{do}\ s]\!] = \mathrm{lfp}(F)$
for $init$; p; $incr$ do s	$F(H)(\sigma) = \begin{cases} \big([\![s]\!](\sigma)\big) \triangleright (\lambda\sigma'.\, [\![incr]\!](\sigma') \triangleright H) & \text{if } [\![p]\!]_\sigma = \mathsf{true}, \\ \mathsf{Norm}(\sigma) & \text{if } [\![p]\!]_\sigma = \mathsf{false} \end{cases}$; $[\![\mathsf{for}\ init; p; incr\ \mathsf{do}\ s]\!](\sigma) = [\![init]\!](\sigma) \triangleright (\lambda\sigma'.\, \mathrm{lfp}(F)(\sigma'))$
do s while p	$F(H)(\sigma) = [\![s]\!](\sigma) \triangleright (\lambda\sigma'. \begin{cases} H(\sigma') & \text{if } [\![p]\!]_{\sigma'} = \mathsf{true}, \\ \mathsf{Norm}(\sigma') & \text{if } [\![p]\!]_{\sigma'} = \mathsf{false} \end{cases})$; $[\![\mathsf{do}\ s\ \mathsf{while}\ p]\!] = \mathrm{lfp}(F)$
return e	$[\![\mathsf{return}\ e]\!](\sigma) = \mathsf{Ret}([\![e]\!]_\sigma, \sigma)$
assert(p), require(p)	$[\![\cdot]\!](\sigma) = \begin{cases} \mathsf{Norm}(\sigma) & \text{if } [\![p]\!]_\sigma = \mathsf{true}, \\ \mathsf{Abort} & \text{if } [\![p]\!]_\sigma = \mathsf{false} \end{cases}$
revert$(\cdots)$	$[\![\mathsf{revert}(\cdots)]\!](\sigma) = \mathsf{Abort}$
try e (returns (x)) s_t catch s_c	$[\![\cdot]\!](\sigma) = \begin{cases} [\![s_t]\!](\sigma[x \mapsto v]) & \text{if call succeeds with } v, \\ [\![s_c]\!](\sigma) & \text{if call reverts} \end{cases}$
call$(\overline{e})$	Internal: parameter binding; external: unspecified

Table B.5: Abstract semantics (denotational)

Statement	**Meaning**
skip	$[\![\mathsf{skip}]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Norm}}(\hat{\sigma})$
$s_1; s_2$	$[\![s_1; s_2]\!]^\sharp(\hat{\sigma}) = \big([\![s_1]\!]^\sharp(\hat{\sigma})\big) \triangleright^\sharp (\lambda\hat{\sigma}'.\, [\![s_2]\!]^\sharp(\hat{\sigma}'))$
$\tau\ x;$	$[\![\tau\ x;]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Norm}}\big(\hat{\sigma}[x \mapsto \hat{\mathrm{init}}(\tau)]\big)$
$\tau\ x = e;$	$[\![\tau\ x = e;]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Norm}}\big(\hat{\sigma}[x \mapsto \alpha_\tau([\![e]\!]^\sharp_{\hat{\sigma}})]\big)$
$lv := e$	$[\![lv := e]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Norm}}\big(\widehat{\mathsf{write}}(\hat{\sigma}, lv, [\![e]\!]^\sharp_{\hat{\sigma}})\big)$
delete lv	$[\![\mathsf{delete}\ lv]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Norm}}\big(\widehat{\mathsf{write}}(\hat{\sigma}, lv, \hat{\mathrm{zero}}_{\tau(lv)})\big)$
if p then s_t else s_f	$\hat{\sigma}_t = \mathrm{refine}(\hat{\sigma}, p, \mathsf{true})$, $\hat{\sigma}_f = \mathrm{refine}(\hat{\sigma}, p, \mathsf{false})$; $[\![\cdot]\!]^\sharp(\hat{\sigma}) = \mathrm{joinRes}\big([\![s_t]\!]^\sharp(\hat{\sigma}_t),\ [\![s_f]\!]^\sharp(\hat{\sigma}_f)\big)$
while p do s	$G^\sharp(H)(\hat{\sigma}) = \mathrm{joinRes}\big([\![s]\!]^\sharp(\mathrm{refine}(\hat{\sigma}, p, \mathsf{true})) \triangleright^\sharp H,\ \widehat{\mathsf{Norm}}(\mathrm{refine}(\hat{\sigma}, p, \mathsf{false}))\big)$; $[\![\mathsf{while}\ p\ \mathsf{do}\ s]\!]^\sharp = \mathrm{lfp}^\nabla(G^\sharp)$
for *init*; p; *incr* do s	$G^\sharp(H)(\hat{\sigma}) = \mathrm{joinRes}\big([\![s]\!]^\sharp(\mathrm{refine}(\hat{\sigma}, p, \mathsf{true})) \triangleright^\sharp (\lambda\hat{\sigma}'.\, [\![incr]\!]^\sharp(\hat{\sigma}') \triangleright^\sharp H),\ \widehat{\mathsf{Norm}}(\mathrm{refine}(\hat{\sigma}, p, \mathsf{false}))\big)$; $[\![\mathsf{for}\ init; p; incr\ \mathsf{do}\ s]\!]^\sharp = [\![init]\!]^\sharp(\hat{\sigma}) \triangleright^\sharp (\lambda\hat{\sigma}'.\, \mathrm{lfp}^\nabla(G^\sharp)(\hat{\sigma}'))$
do s while p	$G^\sharp(H)(\hat{\sigma}) = [\![s]\!]^\sharp(\hat{\sigma}) \triangleright^\sharp (\lambda\hat{\sigma}'.\, \mathrm{joinRes}\big(H(\mathrm{refine}(\hat{\sigma}', p, \mathsf{true})), \widehat{\mathsf{Norm}}(\mathrm{refine}(\hat{\sigma}', p, \mathsf{false}))\big))$; $[\![\mathsf{do}\ s\ \mathsf{while}\ p]\!]^\sharp = \mathrm{lfp}^\nabla(G^\sharp)$
return e	$[\![\mathsf{return}\ e]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Ret}}([\![e]\!]^\sharp_{\hat{\sigma}}, \hat{\sigma})$
assert(p), require(p)	$\widehat{\mathsf{Norm}}(\mathrm{refine}(\hat{\sigma}, p, \mathsf{true}))$ if p must-hold; $\widehat{\mathsf{Abort}}$ if p must-fail; joinRes otherwise
revert($\cdots$)	$[\![\mathsf{revert}(\cdots)]\!]^\sharp(\hat{\sigma}) = \widehat{\mathsf{Abort}}$
try e (returns (x)) s_t catch s_c	$[\![\cdot]\!]^\sharp(\hat{\sigma}) = \mathrm{joinRes}\big([\![s_t]\!]^\sharp(\hat{\sigma}[x \mapsto \top]),\ [\![s_c]\!]^\sharp(\hat{\sigma})\big)$
call($\overline{e}$)	Internal: parameter binding; external: havoc footprint or $\widehat{\mathsf{Abort}}$